

第20章 IP协议

在因特网模型中，主要的网络协议是网际协议（Internet Protocol, IP）。在这一章中，我们首先讨论互联网和网络层协议的一般问题。

然后讨论网际协议当前版本4，即IPv4。进一步使我们提出这个协议的下一代，即IPv6。不久的将来它可能成为主要的协议。

最后，我们讨论从IPv4过渡到IPv6的策略。有些读者可能注意到没有IPv5，因为IPv5是基于从未实现过的OSI模型的实验性协议。

20.1 网际互联

网络的物理层和数据链路层在本地运行。这两层共同负责网络相邻节点间的数据传递，如图20.1所示。

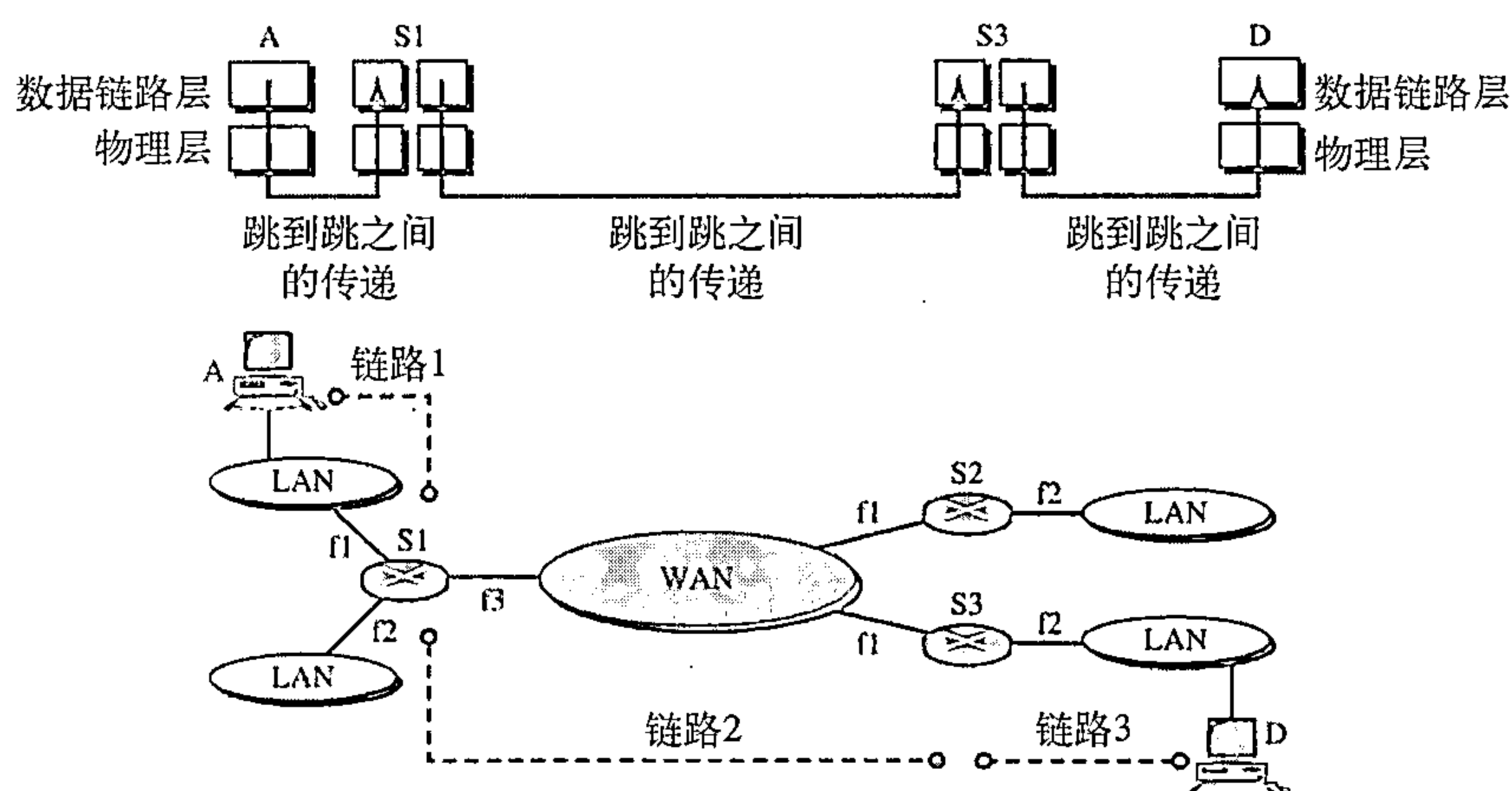


图20.1 两台主机之间的链路

这个互联网是由五个网络组成的：四个局域网和一个广域网。如果主机A需要发送一个数据分组到主机D，该分组首先需要从A到达R1（一个交换机或路由器），然后从R1到达R3，最后从R3到达主机D。我们说数据分组经过三个链路进行传递。在每个链路中，包括两个物理层和数据链路层。

然而，这里有一个很大的问题。当数据到达R1的接口f1时，R1如何知道应该从f3接口将其发送出去？在数据链路层（或物理层）并没有规则来帮助R1做出正确的决策。而且，帧也没有携带任何路由选择信息，帧中只包含主机A的MAC地址（源地址）和R1的MAC地址（目的地址）。对于一个局域网或广域网，传递意味着通过一条而不是多条链路传送帧。

20.1.1 网络层需求

为了解决通过多条链路进行传递的问题，就设计了网络层（有时称之为互联网络层）。网络层不仅负责主机间的传递，而且还负责通过路由器或交换机对分组进行路由选择。图20.2说明增加了一个网络层的同一互联网络。

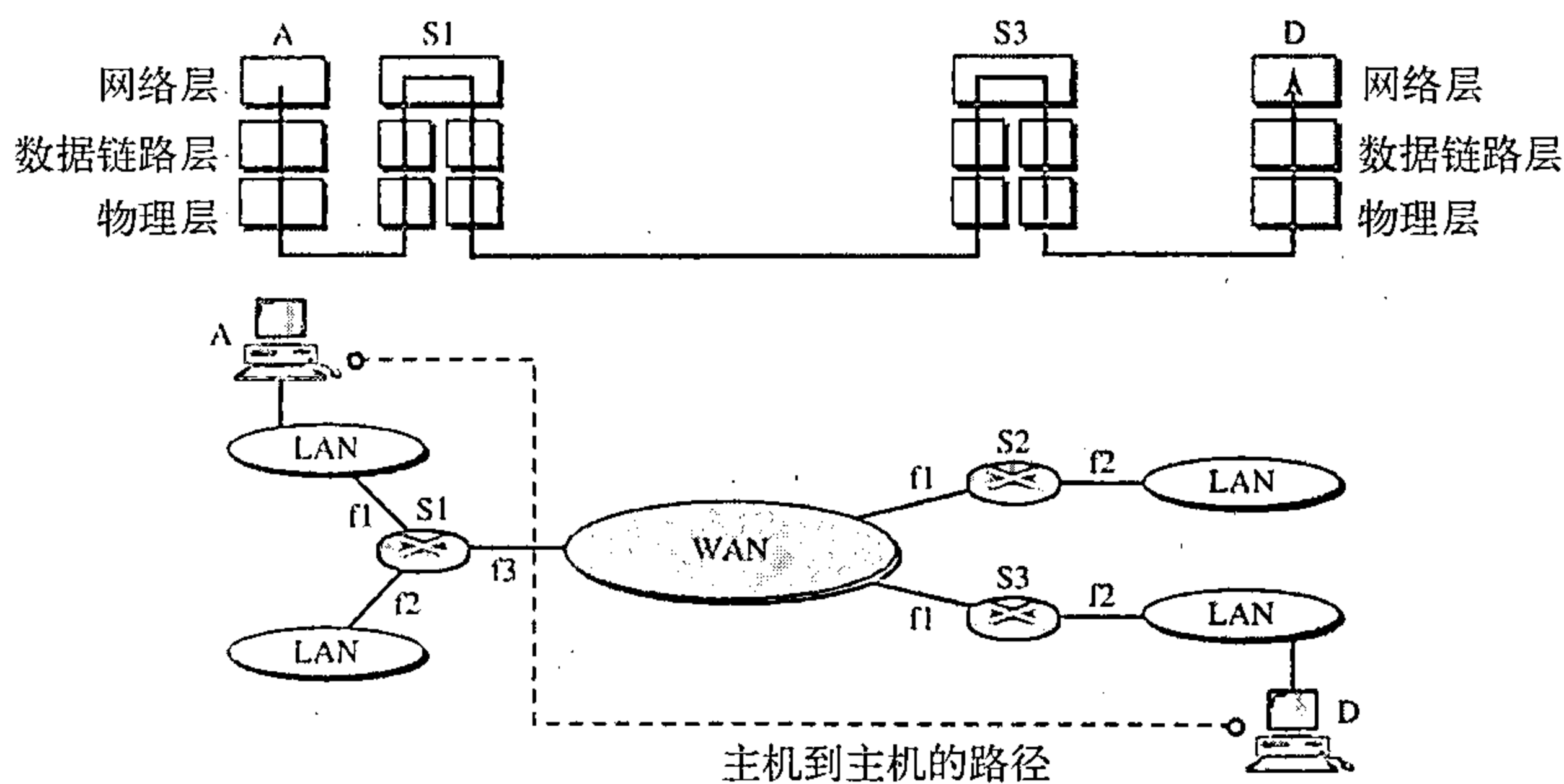


图20.2 互联网中的网络层

图20.3说明了在源端、路由器端和目的端的网络层功能总的思想。在源端网络层负责将来自另一个协议（如一个传输层协议或一个路由协议）的输入数据生成一个分组，分组的头部包含源和目的逻辑地址以及其他信息，网络层负责检验路由表寻找路由选择信息（如分组出去的接口或下一节点的物理地址）。如果分组太大，那么就得对其进行分段（在本章的后面讨论分段）。

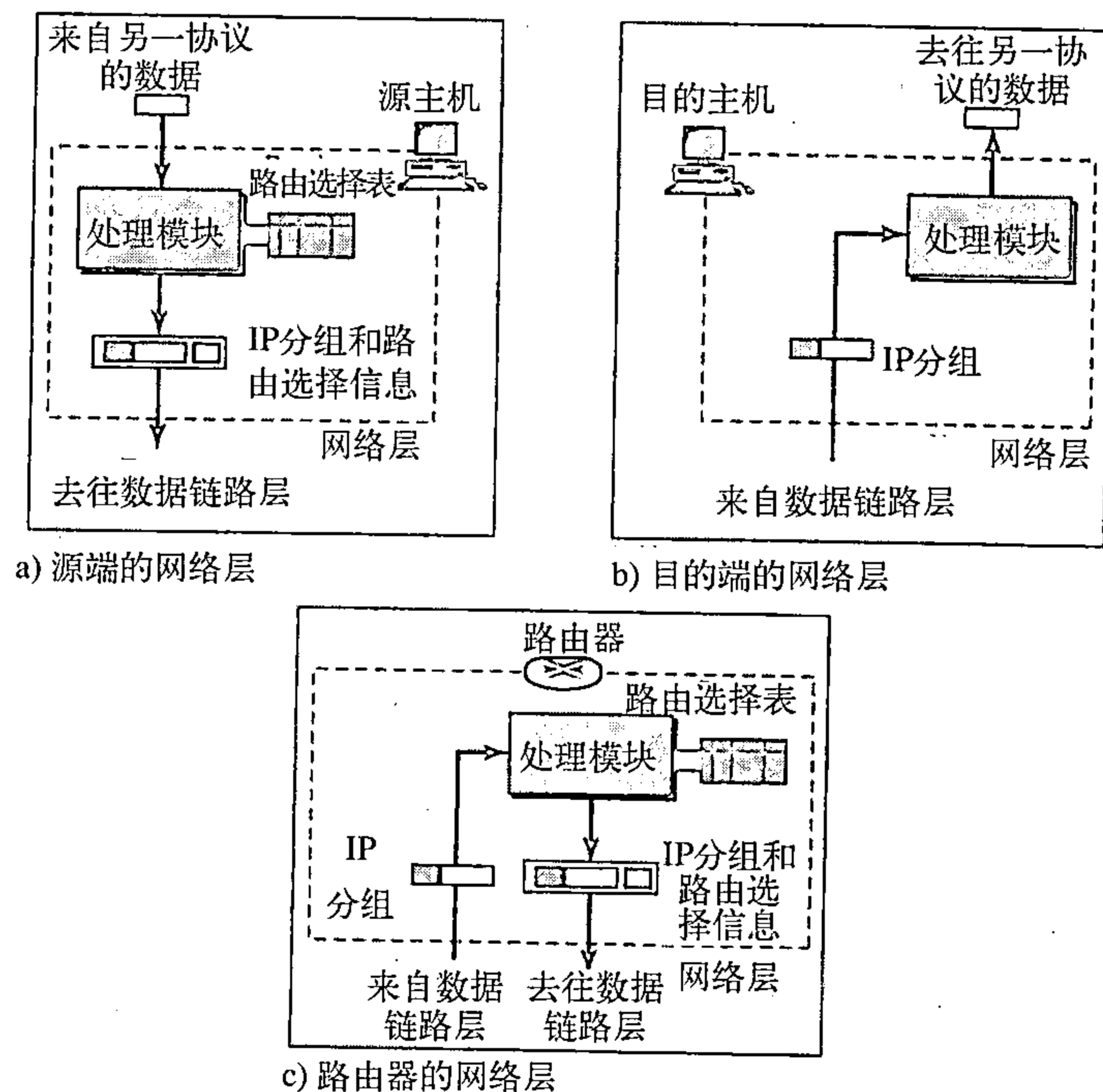


图20.3 源端、路由器端和目的端的网络层

交换机或路由器中的网络层负责对分组进行路由选择。当一个分组到达时，路由器或交换机就从它的路由选择表中为该分组找到一个必须将其发送出去的接口。改变头部的某些内容后的分组按路由选择信息再传送给数据链路层。

目的端的网络层负责地址验证。它确保分组中的目的地址与主机地址是相同的。如果分组是一个分段，网络层就等待所有的分段到达后再对其进行重组，然后再将重组后的分组交给传输层。

20.1.2 作为数据报网络的因特网

因特网的网络层是一种分组交换网络。我们在第8章讨论了交换。我们说，交换可以分为三大类：电路交换、分组交换和报文交换。分组交换使用的是虚电路方法或数据报的方法。

在网络层，因特网使用数据报交换方法。它利用网络层所定义的全球地址从源端到目的端来路由分组。

因特网中的网络层交换是利用数据报分组交换方法来实现的。

20.1.3 作为无连接网络的因特网

分组传递能够利用面向连接的网络服务来实现，也可利用面向无连接的网络服务来实现。在面向连接的服务（connection oriented service）中，源端在发送一个分组之前首先与目的端建立一个连接。在连接建立后，分组就可以按顺序依次从相同的源端发送到相同的目的端。在这种情况下，分组间就存在一种关系，它们沿相同的路径按顺序发送。一个分组与其前后的分组在逻辑上连接在一起。当一个报文的所有分组都传递完毕后，连接就会终止。

在一个面向连接的协议中，当一个连接建立之后，对那些具有相同源和目的地址的分组序列，只会进行一次路由策略。交换机不会为每个单独的分组重复计算路由。这种类型的服务用在类似帧中继和ATM等虚电路分组交换方法中。

在无连接的服务（connectionless service）中，网络层协议独立地对待每个分组，而且每个分组与任何其他分组没有联系。一个报文中分组可能会也可能不会沿同样的路径到达其目的地。这种类型的服务用于数据报分组交换方法中。因特网在其网络层就采用这种类型的服务。

这样做的原因是：因特网是由许多异构网络所组成的，因此在没有提前弄清网络特性的前提下，在源端和目的端建立一个连接几乎是不可能的。

因特网的网络层通信是无连接的。

20.2 IPv4

网际协议第4版（Internet Protocol version4, IPv4）是TCP/IP协议使用的传输机制，图20.4表示了IPv4在TCP/IP协议族中的位置。

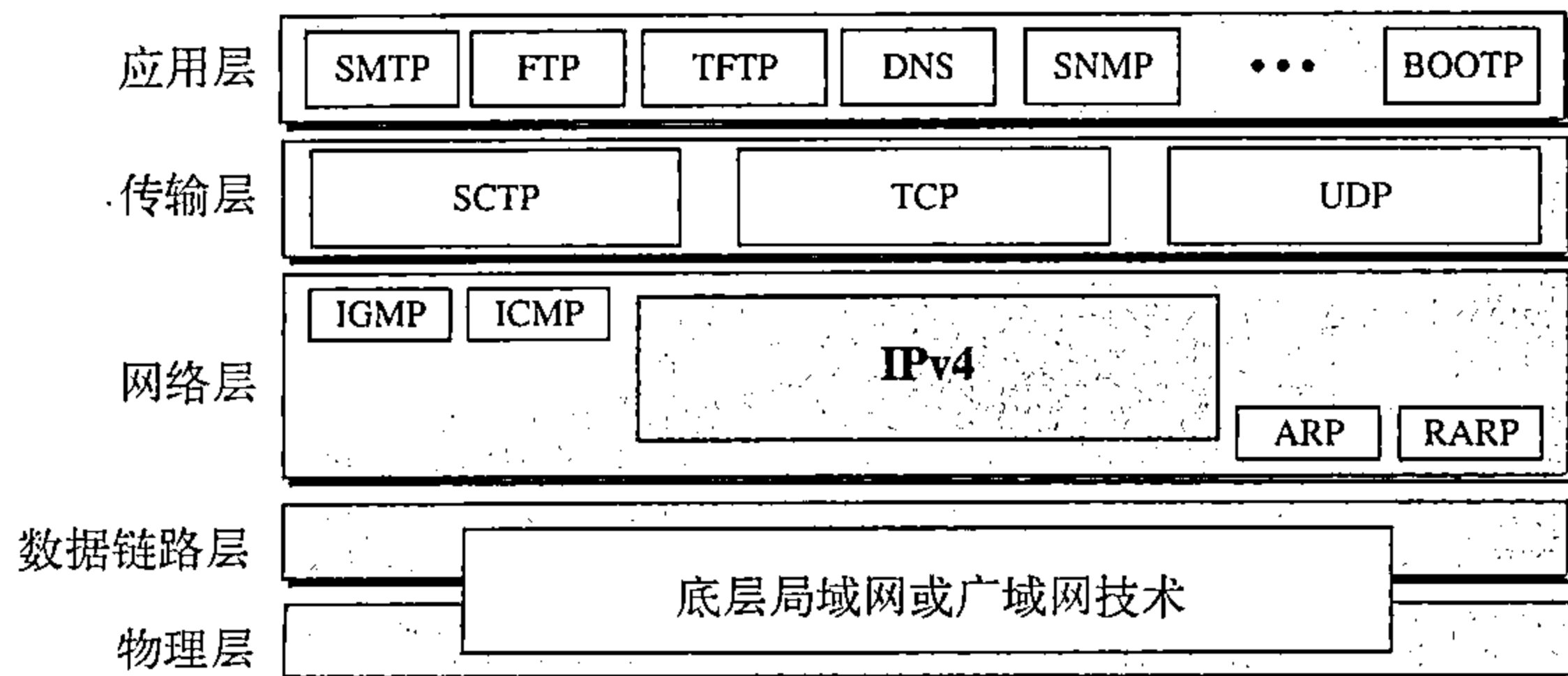


图20.4 IPv4在TCP/IP协议族中的位置

IPv4是一种不可靠的无连接数据报协议——尽力传递（best-effort delivery）。尽力传递这一词是指IPv4协议不提供差错控制或流量控制（除在头部差错检测外）。IPv4假定底层是不可靠的，因此尽力传输到目的地，但没有保证。

当可靠性很重要时，IPv4必须与一个可靠协议（如TCP）配合起来使用。理解尽力传递的

常见例子是邮局。邮局尽力传递邮件，但不是永远成功。如果一封未挂号的信遗失，那只能由发信人或可能的收信人来发现遗失或损坏的情况。

IPv4也是使用数据报的分组交换网的无连接协议（见第8章），这就是说每一分组独立进行处理，而每一分组使用不同的路由传送到目的端。这表明了如果一个源端向同一目的端发送许多数据报，那么这些数据报有可能不按顺序到达。有一些数据报也可能遗失，而有些在传输过程中可能受损坏。此时，IPv4要依靠高层的协议解决这些问题。

20.2.1 数据报

IPv4层的分组称为数据报（datagram），图20.5表示了IPv4数据报的格式。

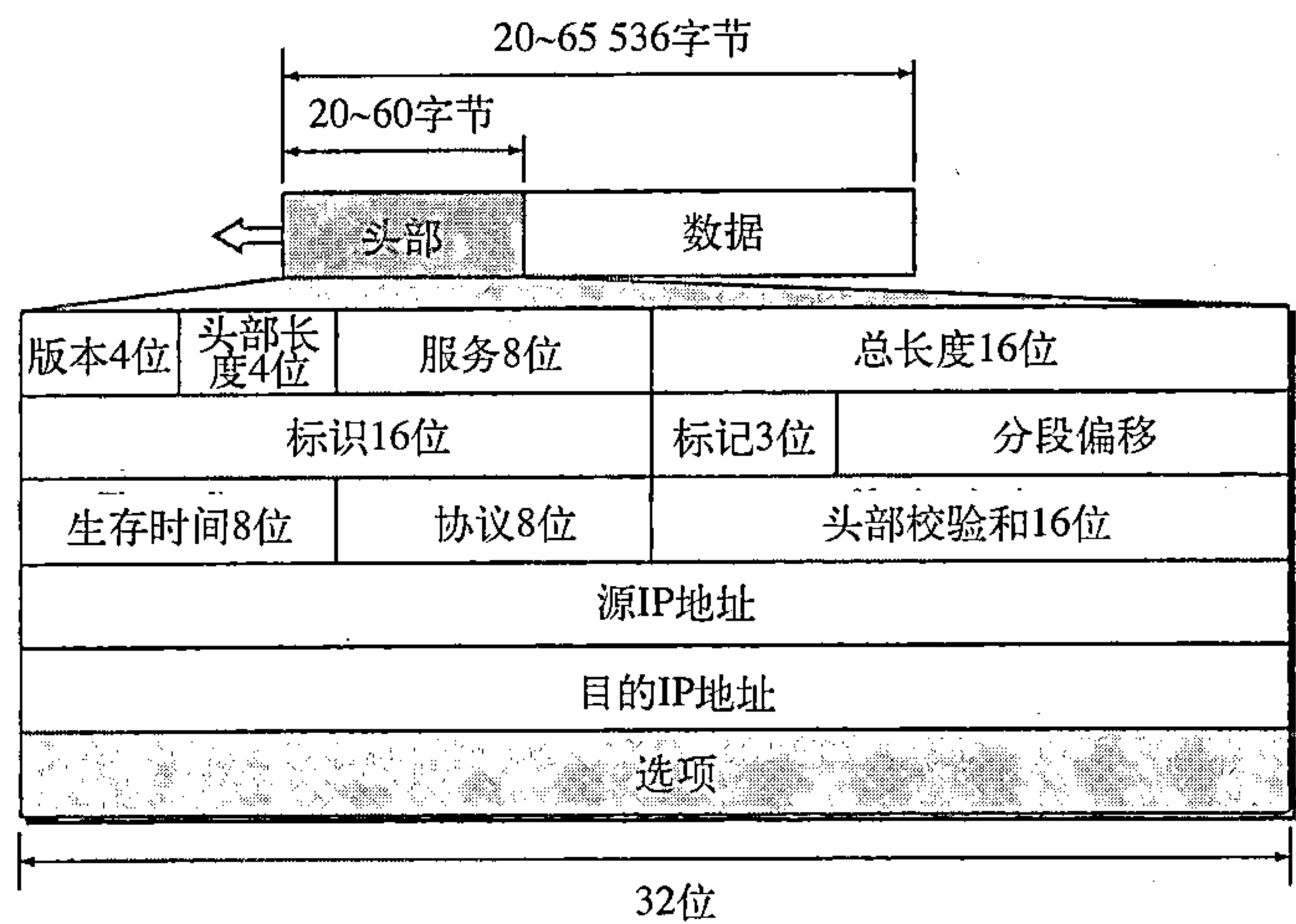


图20.5 IPv4数据报格式

数据报是一个可变长分组，由两个部分组成：头部和数据。头部长度可由20到60个字节组成，包含有与路由选择和传输有关的重要信息。习惯上在TCP/IP中都以4个字节表示头部。按顺序简要地介绍每个字段。

- 版本（VER）。这4位字段定义IPv4版本。目前版本是4，但在不久版本6（IPng）将取代版本4。这个字段向处理机所运行的IP软件指明该IPv4数据报是版本4格式。所有字段都要按版本4的协议来解释。如果计算机使用其他版本，则丢弃数据报而不是错误地解释。
- 头部长度（HLEN）。这4位字段以4字节字定义数据报头部的总长度。这个字段是必需的，因为头部长度是可变的（在20到60个字节之间）。当没有选项时，头部长度是20个字节，而这个字段的值是5（ $5 \times 4 = 20$ ）。当选项字段为最大值时，这个字段的值是15（ $15 \times 4 = 60$ ）。
- 服务。IETF已经改变了这个8位字段的解释与名称，这个字段以前称为服务类型（service type）现称为差分服务（differentiated service）。图20.6显示了这两种解释。

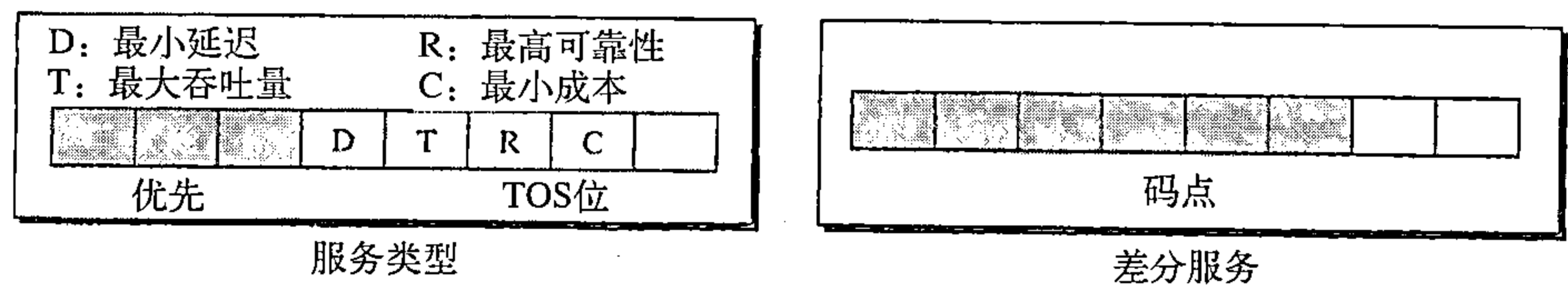


图20.6 服务类型或差分服务

1. 服务类型

在这种解释中，前面的3位称为优先位，而后面的4位称为服务类型（type of service, TOS），最后1位未使用。

a. 优先（Precedence）是3位的子字段，其值由0（二进制000）到7（二进制111）。优先子字段定义了出现如拥塞等问题时，数据报的优先级。当路由器出现拥塞并必须丢弃一串数据报时，最低优先值的数据报将首先被丢弃。在因特网中，有一些数据报比另一些更为重要。例如，用于网络管理的数据报要比向一组人发送的可选信息的一些数据报紧迫得多和重要得多。

在版本4中未使用优先子字段。

b. TOS位是一个4位的子字段，每一位都有其特殊的意义。虽然每一位可以是1或0，但在一个数据报中的这4个位中，只能有一位的值为1，表20.1给出了位模式及其解释。每次仅当一位置1，有5种不同的服务。

应用程序可以请求一个特定类型的服务，但某些应用的默认值，如表20.2所示。

表20.1 服务类型

TOS位	说 明
0000	正常（默认）
0001	最小成本
0010	最高可靠性
0100	最大吞吐量
1000	最小延迟

表20.2 服务类型的默认值

协 议	TOS位	说 明	协 议	TOS位	说 明
ICMP	0000	正常	FTP（控制）	1000	最小延迟
BOOTP	0000	正常	TFTP	1000	最小延迟
NNTP	0001	最小成本	SMTP（命令）	1000	最小延迟
IGP	0010	最高可靠性	SMTP（数据）	0100	量大吞吐量
SNMP	0010	最高可靠性	DNS（UDP查询）	1000	最小延迟
TELNET	1000	最小延迟	DNS（TCP查询）	0000	正常
FTP（数据）	0100	最大吞吐量	DNS（区域）	0100	最大吞吐量

从表20.2中可清楚地看到交互式的活动、需要立即引起注意的活动与需要立即响应的活动都要求最小的延迟。发送块数据的活动要求最大吞吐量。管理活动要求最高的可靠性，后台活动要求最小成本。

2. 差分服务

在差分服务情况下，前6位组成码点（codepoint）子字段，而后2位不用。码点子字段可有两种使用方法：

a. 当最右边3位都是0时，最左边3位与服务类型解释中的优先位相同。换言之，它与原来的解释相同；

b. 当最右边3位不全为0时，则6位由因特网或本地机构按表20.3赋予的优先级定义64种服务。第一类包含32种服务类型，第二与第三类分别包含16种服务。第一类（0，2，4，…，62）由因特网工程任务组（IETF）分配。第二类（3，7，11，15，…，63）由本地组织机构使用。第三类（1，5，9，…，61）是临时的用作实验目的。注意：有些数不出现在类中，它们是第一类中的0到31；第二类中的32到47；第三类中的48到63。由于XXX000（包括0，8，16，24，32，40，48，56）

表20.3 码点的值

类	码 点	分配机构
1	XXXXXX0	因特网
2	XXXXX11	本地的
3	XXXXX01	临时的或实验的

属于所有这3大类，这与TOS解释不相容。相反的，在这种分配方法中这些服务属于第一类。注意：这些分配还未最后定下来。

- 总长度。这个16位字段定义了一个以字节计的IPv4数据报的总长度（头部加上数据）。为了找到从上层来的数据长度，可以将总长度减去头部长度的值乘以4就可得到头部长度的值。

$$\text{数据长度} = \text{总长度} - \text{头部长度的值} \times 4$$

因为这个字段是16位长，因此IPv4数据报长度限制在65 535 ($2^{16}-1$) 字节，其中头部占20到60字节，剩下的是从上层来的数据。

总长度定义了包括头部在内的数据报的总长度。

一个长度为65 535字节的数据报在今天的技术看来很长，但在不久的将来数据报的长度可能会增大。因为底层技术允许具有更大的吞吐量（更高带宽的网络）。

当我们在下一节讨论分段时，将会看到某些网络不能将65 535字节的数据报封装成它们的帧。要通过这些网络就必须将这些数据报进行分段。

人们可能会问为什么需要这个字段。当一台机器（路由器或主机）接收到一个帧时，它除去头部和尾部并转发此数据报。那么为什么要使用一个并不需要的字段？回答是：在许多情况下，我们的确不需要这字段的值。但有些情况下，封装在一个帧中的并不仅仅是数据报。例如，以太网协议对能够封装在一个帧中的数据有最小值和最大值的限制（46到1 500个字节）。当数据报的长度小于46字节时，它必须增加一些填充字节才能满足这个要求。在这种情况下，当机器将数据分离出来时，它必须检查数据报的总长度，以便确定实际的数据长度和填充字节的长度（见图20.7）。

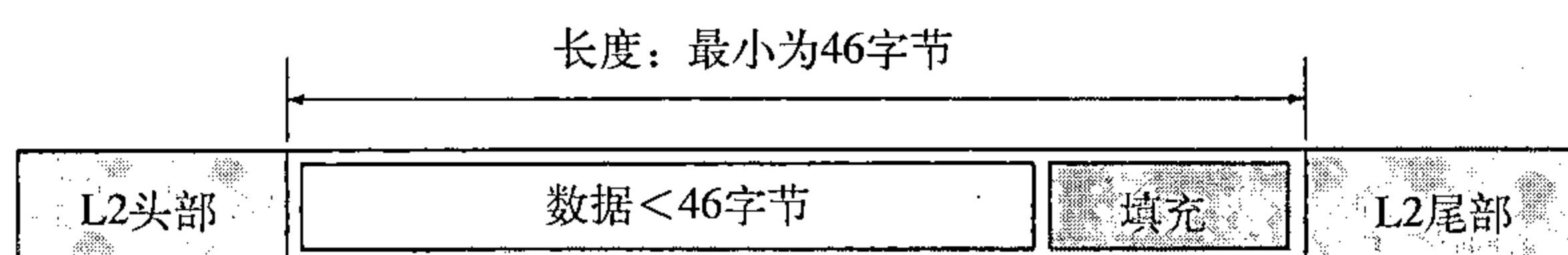


图20.7 一个小的数据报封装在以太网帧中

- 标识。这个字段用于分段中（在下一节讨论）。
- 标记。这个字段用于分段中（在下一节讨论）。
- 分段偏移。这个字段用于分段中（在下一节讨论）。
- 生存时间。一个数据报在它通过互联网时必须具有有限的寿命。这个字段最初设计时是打算保持一个时间戳，由每一个经过的路由器将此值减1。当时间戳的值变为0时，即丢弃该数据报。但是，对于这种方案，所有机器必须是同步的，并且还必须知道一个数据报从一个机器到另一个机器所花费的时间。现在，这个字段基本上是用来控制一个数据报所通过路由器的最大跳数（路由器数）。当源端发送数据报时，它在此字段存入一个数。这个数值约为任何两个主机之间的路由器数量的两倍。处理数据报的每一个路由器将此数值减1。当路由器接收到一个数据报时，它先将此字段的值减1。如果减1之后此字段的值为0，路由器就丢弃该数据报。

这个字段是必需的，因为因特网中的一些路由表可能会出故障。如果发生故障，一个数据报就可能在两个或更多的路由器之间传输很长时间，但永远传输不到目的端。这个字段限制了数据报的寿命。

这个字段的另一个用途是故意限制分组的行程。例如，如果源端想将分组限制在局域范

围内，就可将此字段置为1。当分组到达第一个路由器时，这个值就减为0，因而数据报就被丢弃了。

- 协议。这个8位字段定义了使用此IPv4层服务的高层协议。来自高层一些协议如TCP、UDP、ICMP和IGMP等的的数据能够封装到IPv4数据报中。这个字段指明IPv4数据报必须传递到的最终的目的协议。换言之，由于IPv4协议携带来自高层不同协议的数据，这个字段的值对接收方的网络层了解数据属于哪个协议很有帮助（见图20.8）。

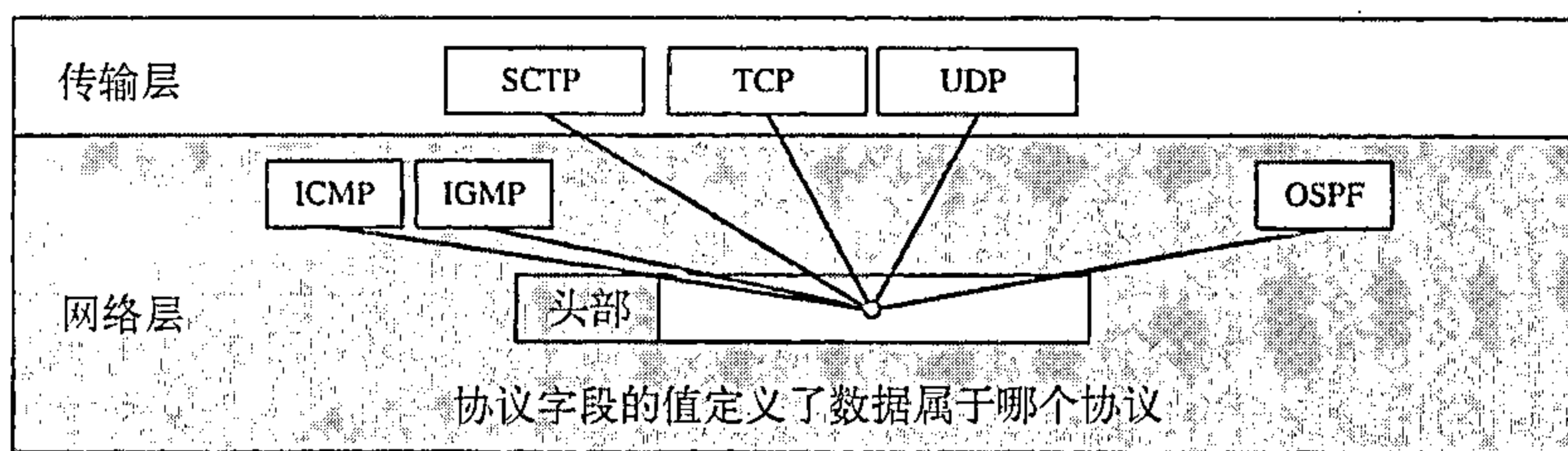


图20.8 协议字段和封装的数据

对于不同的高层协议，这个字段的取值如表20.4所示。

表20.4 协议值

值	协议
1	ICMP
2	IGMP
6	TCP
17	UDP
89	OSPF

- 校验和。校验和的概念及其计算将在本章的后面讨论。
- 源地址。这个32位的字段定义了源端的IPv4地址，在IPv4数据报从源主机到目的主机传输期间，这个字段必须保持不变。
- 目的地址。这个32位的字段定义了目的端的IPv4地址，在IPv4数据报从源主机到目的主机传输期间，这个字段必须保持不变。

例20.1

到达的一个IPv4分组的前8位如下：

01000010

接收方是否应丢弃该分组？为什么？

解：

这个分组有错误，其中最左的4位是版本，它是正确的。下一个4位是1000，则表明它是一个无效长度（ $2 \times 4 = 8$ ）。而头部的最小字节数是20。因此，这个分组在传输过程中被损坏了。

例20.2

在一个IPv4分组中，头部长度字段的值用二进制表示为1000，试问这个分组携带的选项是几个字节？

解：

头部长度值是8，说明头部的总字节数为 $8 \times 4 = 32$ ，前面的20个字节是基本头部，后面的12个字节是选项。

例20.3

在一个IPv4分组中，头部长度字段的值是5，而总长度字段的值是0x0028，试问这个分组携带的数据是多少字节？

解：

头部长度的值为5，就是说头部的总字节数为 $5 \times 4 = 20$ （无选项）。总长度是40字节，也就是说，这个分组携带 $40 - 20 = 20$ 个字节的数据。

例20.4

一个IPv4分组已到达，最前面几个十六进制数字如下：

0x45000028000100000102...

在丢弃这分组之前，它经历了多少跳？数据是属于上层的哪一个协议？

解：

为了求生存时间字段，我们跳过8个字节（16个十六进制数字）。生存时间字段是第9个字节，它的值是01，这就是说分组仅能跳一次。协议字段是下一个字节02，也就是说上层协议是IGMP（见表20.4）。

20.2.2 分段

一个数据报可以通过几个不同的网络进行传输。每一个路由器将它所接收的帧拆封成IPv4数据报，对它进行处理，然后再将它封装成另一个帧。接收到的帧的格式和长度取决于此帧刚刚经过的物理网络所使用的协议；发出去的帧的格式和长度则取决于此帧将要经过的物理网络所使用的协议。例如，如果一个路由器将一个局域网连接到一个广域网，那么它接收到的帧是一个局域网格式的帧，发出去的帧是广域网的格式。

最大传输单元（MTU）

每一个数据链路层协议都有其自己的帧格式，这格式中定义的一个字段是数据字段的最大长度。换言之，当数据报封装成帧时，该数据报的总长度必须小于这个最大数据长度。这是由网络所使用的硬件和软件给出的限制所定义的（见图20.9）。

MTU的值取决于物理网络协议。表20.5显示了不同协议的MTU值。

表20.5 某些网络的MTU值

协 议	MTU
超级通道（Hyperchannel）	65 535
令牌环（16Mbps）	17 914
令牌环（4Mbps）	4 464
FDDI	4 352
以太网	1 500
X.25	576
PPP	296

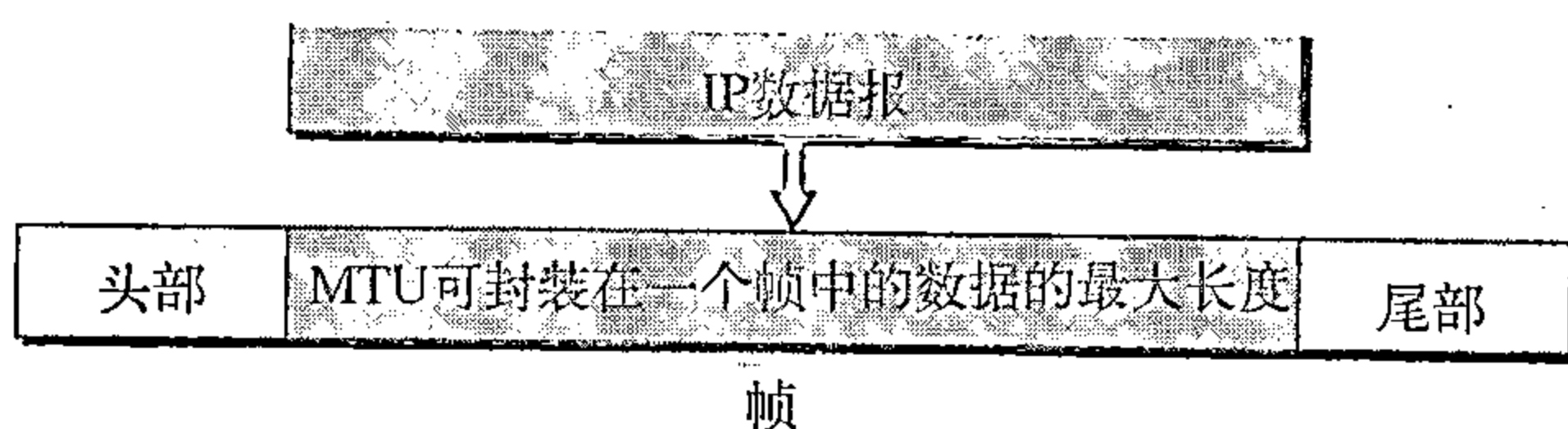


图20.9 最大传输单元（MTU）

为了使IPv4协议与物理网络无关，协议设计者决定使IPv4数据报的最大长度等于最大传输单元，其目前定义值是65 535字节。如果使用具有此值的MTU协议，那么可使传输更加有效。然而，对其他一些物理网络，我们就要将数据报进行分割，使其能够通过这些网络，这个过程称为分段（fragmentation）。

源端通常不对IPv4分组进行分段，传输层将数据分割成能适合在数据链路层所用IPv4的长度。

当对一个数据报进行分段时，每个分段都有其自己的头部，其中大部分的字段是重复的，但有些发生了变化。如果一个已分段的数据报遇到一个更小MTU的网络，那么已分段的数据报还可再进行分段。换句话说，数据报在到达最终的目的端之前，可能经过多次分段。

在IPv4中,数据报可能被主机或其路径中的任何路由器进行分段。然而,数据报的重组只能在目的主机上进行,因为每个分段都是一个独立的数据报。由于被分段的数据报可以沿着不同的路径传输,因此我们不能控制或保证一个分段的数据报会走哪一条路径,而属于同一数据报的所有分段最后都必须到达目的主机。所以,从逻辑上说应当在最终的目的端进行重组。一个更强的目的是在传输期间重组分组将会带来效率的降低。

当对数据报进行分段时,头部中的某些部分需要被复制到所有的分段中。如下一节中所见,选项字段可以被复制,也可以不被复制。对一个数据报进行分段的主机或路由器必须改变三个字段的值:标志、分段偏移和总长度。当然,不管是否进行分段,校验和的值总是重新计算的。

与分段相关的字段

与IPv4数据报的分段和重组相关的字段是:标识、标记和分段偏移。

- 标识。这个16位字段标识一个从源主机发出的数据报。当数据报离开源主机时,这个标识与源IPv4地址必须唯一地定义这个数据报。为了保证唯一性,IPv4协议使用一个计数器来标识数据报。当IPv4协议发送数据报时,就将该计数器的当前值复制到标识字段中,并将此计数器的值加1。只要此计数器保存在主存储器中,唯一性就得到保证。当数据报分段时,标识字段的值就复制到所有的分段中。换言之,所有的分段都有与原始的数据报相同的标识号。该标识号有助于在目的端重组数据报。目的端知道将所有具有相同标识值的分段必须重组成一个数据报。
- 标记。这是一个3位的字段。第一位保留为以后用。第二位称为“不分段位”,如果其值是1,则机器就不能将该数据报进行分段。如果无法将此数据报通过任何可用的物理网络对数据报进行传递,则它就丢弃该数据报,并向源主机发送一个ICMP差错报文(见第21章);如果其值为0,则根据需要对数据报进行分段。第三位称为“多分段位”。如果其值为1,则表示此数据报不是最后的分段,在该分段后还有更多的分段;如果其值为0,则表示它是最后一个或唯一的分段(见图20.10)。
- 分段偏移。这个13位的字段表示这个分段在整个数据报中的相对位置。它是在原始数据报中的数据偏移量,以8字节为度量单位。图20.11说明了一个具有4 000字节的数据报被划分为三个分段。

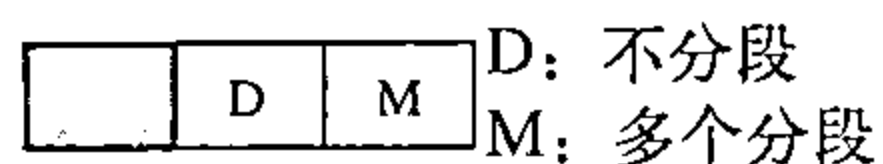


图20.10 标记字段

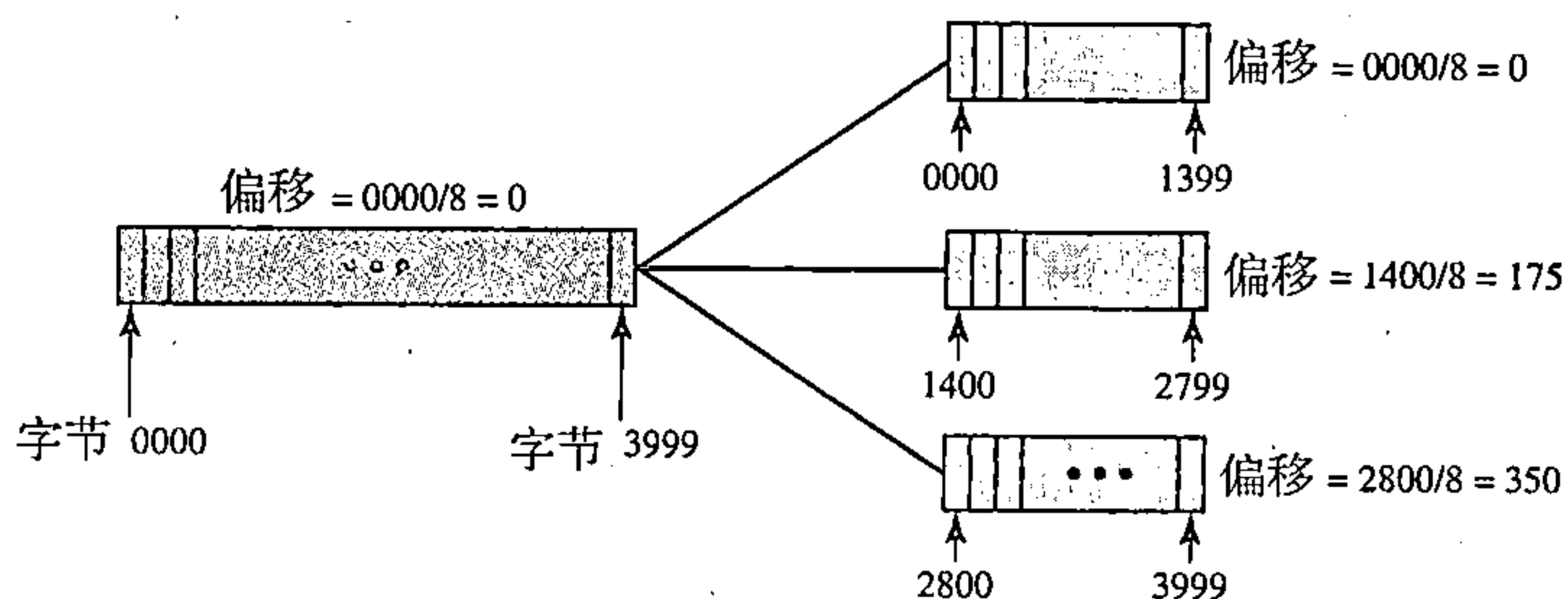


图20.11 分段示例

原始数据报的字节编号从0~3 999,第一个分段携带的数据是字节0~1 399,对于这个数据报,分段的偏移量为0/8=0;第二分段携带的数据是字节1 400~2 799,对于这个数据报其偏移量为1 400/8=175。最后,第三个分段携带的数据是字节2 800~3 999,其偏移量为2 800/8=350。

请记住，偏移量是以8字节为单位的。这样做是因为偏移字段的长度只有13位，不能用来表示一个大于8 191的字节数。因此，将数据报进行分段的主机或路由器必须这样选择每个分段的长度，即第一个字节编号能被8整除。

图20.12表示了图20.11中分段的扩展图。注意：所有分段的标识字段的值都相同。还要注意，除最后一个分段外，所有分段的标记中的值都是设置为多个分段。每一个分段的偏移字段的值也在图中表示。

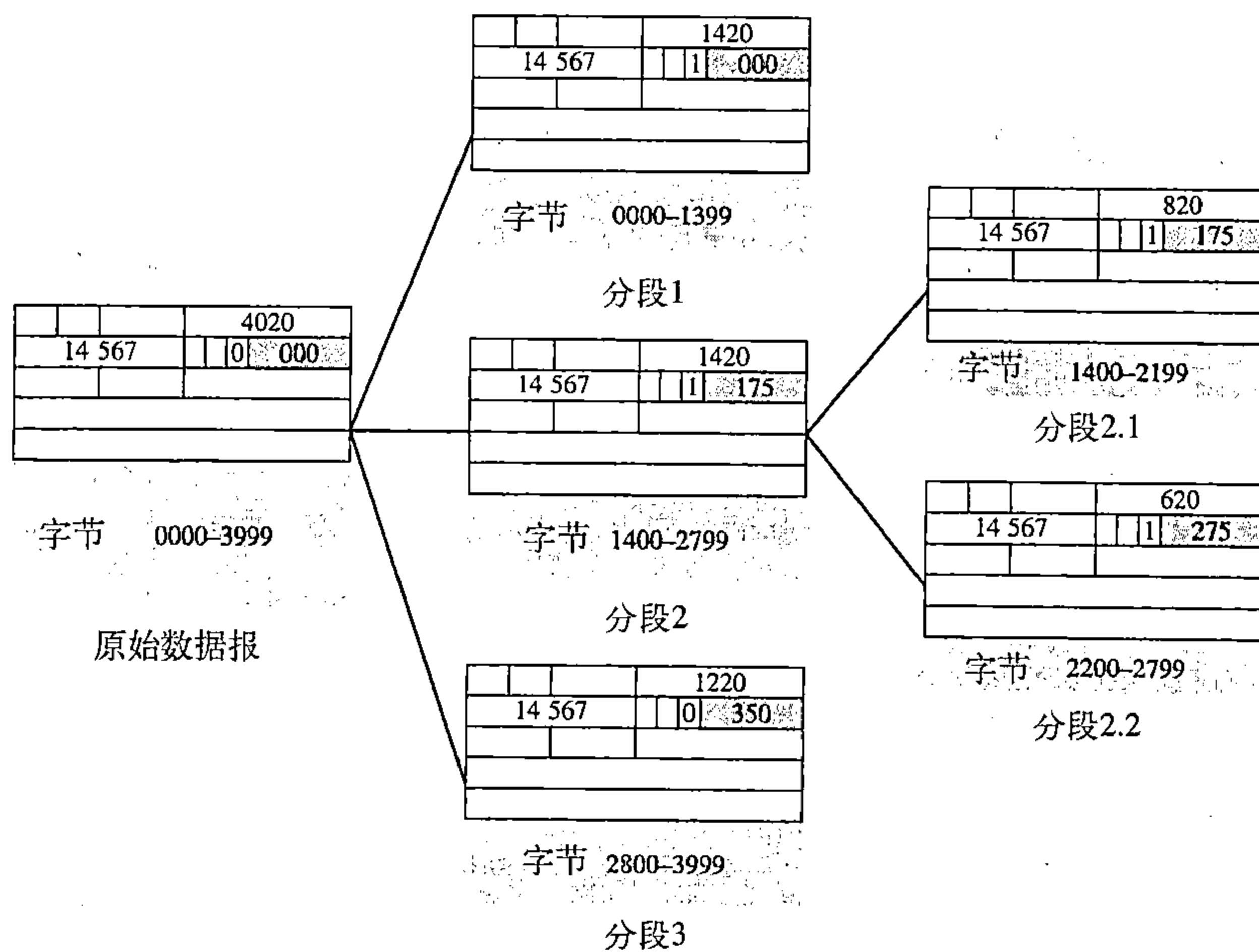


图20.12 分段的细节示例

图20.12还表示了一个分段本身再进行分段时会发生什么。在这种情况下，分段偏移值永远是相对于原始的数据报。例如，图20.12中的第二个分段又划分为两个长度分别为800字节和600字节的分段，但这些分段的偏移量所表示的位置都相对于原始的数据。

显然，即使每一个分段走不同的路径，并在到达时失序，最终目的主机也能用收到的这些分段（假定没有丢失）重组原始数据报。所使用的策略如下：

1. 第一个分段的偏移字段的值为0；
2. 将第一个分段长度除以8，其结果就是第二个分段的偏移值；
3. 将第一个和第二个分段的总长度除以8，其结果为第三个分段的偏移值；
4. 继续以上过程，最后一个分段的多个分段位的值为0。

例20.5

到达的一个分组的M位的值是0，试问这是第一个分段还是中间的分段，还是最后的分段？我们是否知道这个分组已被分段？

解：

如果M位是0，这就是说不存在更多的分段，该分段是最后的一个分段。但是我们不能说原来的分组是否已被分段。没有分段的分组被认为是最后一个分段。

例20.6

到达的一个分组的M位的值是1，试问这是第一个分段还是中间的分段，还是最后的分段？我们是否知道这个分组已被分段？

解:

如果M位是1, 这就是说至少还有一个分段, 这个分段是第一个或中间的分段, 而不是最后的分段。但我们不知道它是第一个分段还是中间分段, 还需要有更多的信息(分段偏移值), 见例20.7。

例20.7

到达的一个分组的M位的值是1, 而偏移值是0, 试问这是第一个分段还是最后的分段, 或是中间的分段?

解:

因为M位是1, 它或是第一个分段或是中间分段。由于偏移值为0, 因此它是第一个分段。

例20.8

到达的一个分组的偏移值是100, 试问第一个字节的编号是什么? 我们能知道最后一个分段的编号吗?

解:

为了求第一个字节的编号, 我们将偏移值乘8, 这就说第一个字节的编号是800。但我们不能确定最后字节的编号, 除非知道数据的长度。

例20.9

到达的一个分组的偏移值是100, 而HLEN字段值为5, 总长度字段的值是100。试问第一个字节和最后字节的编号是多少?

解:

第一个字节的编号是 $100 \times 8 = 800$, 总长度是100个字节, 头部长度是20个字节 (5×4), 这就是说这个数据报有80个字节。如果第一个字节的编号是800, 则最后字节的编号是879。

20.2.3 校验和

在第10章中, 我们已讨论过校验和总的思想及其计算方法。在IPv4分组中的校验和的计算方法与之相同。首先, 将校验和字段置为0。然后, 将整个头部划分为16位的部分, 并将各部分相加。将计算结果(和)取反码, 插入到校验和字段中。

IPv4分组中的校验和只对头部进行, 而不在数据部分进行。这有两点很充足的理由。首先, 所有将数据封装在IPv4数据报中的高层协议中, 都有覆盖整个分组的校验和。因此, IPv4数据报的校验和就不必校验所封装的数据部分。其次, 每经过一个路由器, IPv4数据报的头部就要改变一次, 但数据部分不改变。因此, 校验和只对发生变化的部分进行校验。如果包含数据部分, 则每一路由器必须重复计算整个分组的校验和, 这就表示每一个路由器要花费更多的处理时间。

例20.10

图20.13表示了一个对没有选项的IPv4首部计算校验和。首部分成多个16位的部分。将所有这些部分相加, 再将得到的和取反码。将其结果插入到校验和字段中。

20.2.4 选项

IPv4数据报的头部由两部分组成: 固定部分与可变

4	5	0	28
1	0	0	
4	17	0	
10.12.14.5			
12.6.7.9			

4, 5和0	→	4	5	0	0
28	→	0	0	1	C
1	→	0	0	0	1
0和0	→	0	0	0	0
4和17	→	0	4	1	1
0	→	0	0	0	0
10.12	→	0	A	0	C
14.5	→	0	E	0	5
12.6	→	0	C	0	6
7.9	→	0	7	0	9
和	→	7	4	4	E
校验和	→	8	B	B	1

图20.13 IPv4中校验和的计算

部分。固定部分的长度是20个字节，已在前一节讨论过。可变部分由若干选项组成，它最长可达40个字节。

选项正如其名字所隐含的，它对每个数据报来说并不是必需的。这些选项用于网络测试和调试。虽然选项并非IPv4数据报必需的部分，但选项处理却是IPv4软件的必需部分。这表示所有的标准必须能够处理选项，如果这些选项出现在头部中。

每个选项详细的讨论已超出本书的范围。在图20.14中给出选项的分类，并对每项进行了简要地说明。

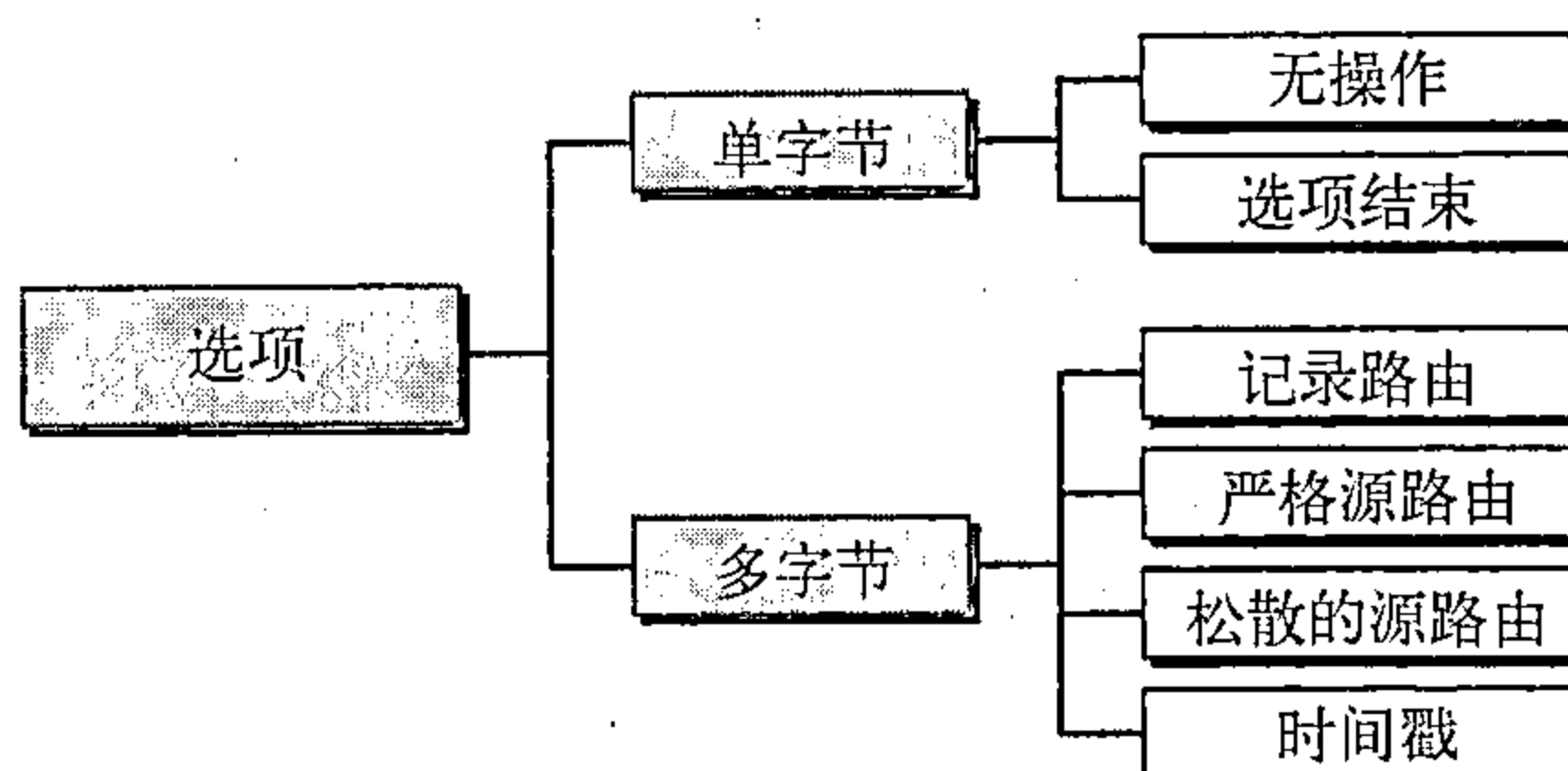


图20.14 IPv4中选项的种类

无操作

无操作选项（no-operation option）是一个1字节选项，用做选项之间的填充符。

选项结束

选项结束选项（end-of-option option）也是一个1字节，用于选项字段结束时的填充。但是，它只能用做最后一个选项。

记录路由

一个记录路由选项（record route option）是用来记录处理数据报的因特网路由器。它可列出多达9个路由器地址，它用于调试和管理目的。

严格源路由

源端使用严格源路由选项（strict source route option），用来预先确定数据报在因特网中传送时的路由。由于几个原因，源端确定路由是很有用的。发送方可以选择一个具有特定服务类型的路由，如最小延迟或最大吞吐量。此外，出于发送端的目的，它还可选择一条更加安全或更加可靠的路由。例如，发送方可以选择某条路由使得数据报不经过对手的网络。

如果一个数据报指明了一个严格的源路由，该数据报就必须经过选项中定义的所有路由器。若路由器的IPv4地址未列入到数据报中，则该数据报就一定不能通过此路由器。若一个数据报通过一个未列入的路由器，则该路由器将数据报丢弃并发出差错报文。

松散的源路由

松散的源路由选项（loose source route option）与严格源路由选项相似，但更加宽松。必须访问表中的路由器，但数据报还可以访问其他的路由器。

时间戳

时间戳选项（timestamp option）用来记录路由器处理数据报的时间。时间是世界时，从午夜开始以毫秒计。知道一个数据报的处理时间将有助于用户和管理者对因特网上的各个路由器的行为进行跟踪。我们能够估计一个数据报从一个路由器到另一个路由器所需的时间。我们说估计，是因为虽然所有的路由器都使用世界时，但它们的本地时钟可能没有进行同步。

20.3 IPv6

TCP/IP协议族中的网络层协议现在是IPv4（网际协议，版本4），IPv4提供在因特网中的系统之间的主机到主机的通信。虽然IPv4设计得很好，但自从20世纪70年代IPv4问世以来，数据通信已经有了很大的发展。IPv4有一些缺点（列出如下），这使得它对飞速发展的因特网

有些不适应。

- 虽然有许多近期的解决方案，如子网化、无类寻址和NAT，但在因特网中地址耗尽还依旧是一个长期的问题。
- 因特网必须能适应实时音频和视频传输。这种类型的传输要求最小延迟的策略和预留资源，而这些在IPv4的设计中并没有提供。
- 对于某些应用，因特网必须能够对数据进行加密和鉴别，IPv4不提供数据的加密和鉴别。

为了克服这些缺点，IPv6（网际协议，版本6，Internetworking Protocol, version 6），也称为下一代网际协议（IPng, Internetworking Protocol, next generation）被提出，并已成为标准。在IPv6中，网际协议修改了很多，以便适应因特网不可预见的增长。IP地址格式和长度以及分组的格式都改变了。相关的一些协议，如ICMP也修改了。网络层的其他一些协议，如ARP、RARP和IGMP，则被取消或包含在ICMPv6协议之中（见第21章）。路由选择协议，如RIP和OSPF（见第22章）也进行了少量的修改以适应变化。通信专家预计，IPv6及其相关协议将很快地取代当前的IP版本。在本节中，将先讨论IPv6，然后研究从IPv4过渡到IPv6的一些策略。

对IPv6的采用进展很慢，理由是最初的动机为它的发展，IPv4地址消耗尽，已经被短期策略修改，如无类寻址和NAT。但是，因特网快速应用的发展和新的服务的出现，如移动IP、IP电话和IP移动电话最终要求用IPv6全部代替IPv4。

20.3.1 优点

与IPv4相比，下一代IP或IPv6具有如下的优点：

- 更大的地址空间。正如第19章中讨论的那样，IPv6地址是128位长。与32位地址相比，其地址空间增加了很多（ 2^96 ）。
- 更好的头部格式。IPv6使用了新的头部格式，其选项与基本头部分开，如果需要，可将选项插入到基本头部与上层数据之间。这就简化和加速了路由选择过程，因为大多数的选项不需要由路由器检查。
- 新的选项。IPv6有一些新的选项来实现附加的功能。
- 允许扩充。如果新的技术或应用需要时，IPv6允许协议进行扩充。
- 支持资源分配。在IPv6中，服务类型字段被取消了，但增加了一种机制（称为流标号）使得源端可以请求对分组进行特殊的处理。这种机制可用来支持像实时音频和视频的通信量。
- 支持更多的安全性。在IPv6中的加密和鉴别选项提供了分组的保密性和完整性。

20.3.2 分组格式

IPv6分组格式如图20.15所示，每个分组由必须要有的基本头部和紧接其后的有效载荷所组成。有效载荷由两部分组成：可选的扩展头部和来自上层的数据。基本头部占用40字节，而扩展头部和来自上层的数据可以包含多达65 535字节的信息。

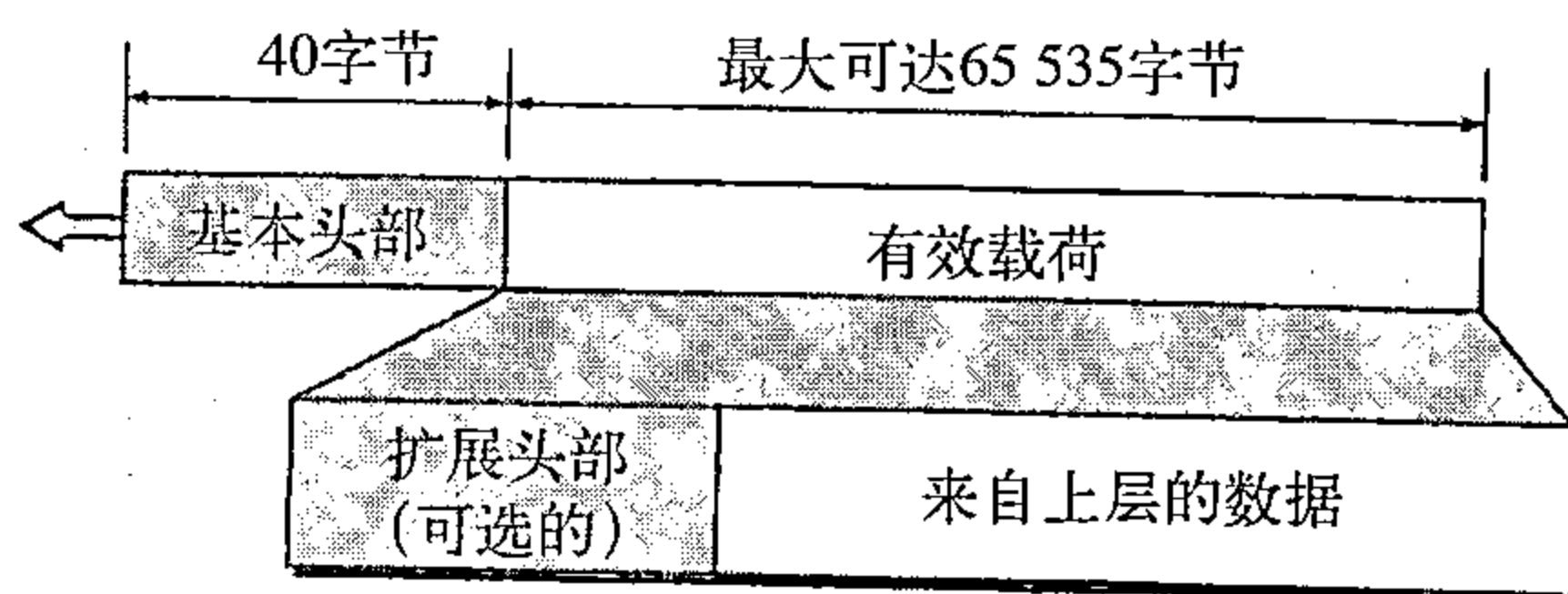


图20.15 IPv6数据报头部和有效载荷

基本头部

图20.16说明了具有8个字段的**基本头部**（base header）。

这些字段如下：

- 版本。这个4位字段定义了IPv6的版本号。对IPv6，其值为6。
- 优先级。这个4位字段定义了当发生通信量拥塞时的分组的优先级。我们将在后面讨论这个字段。

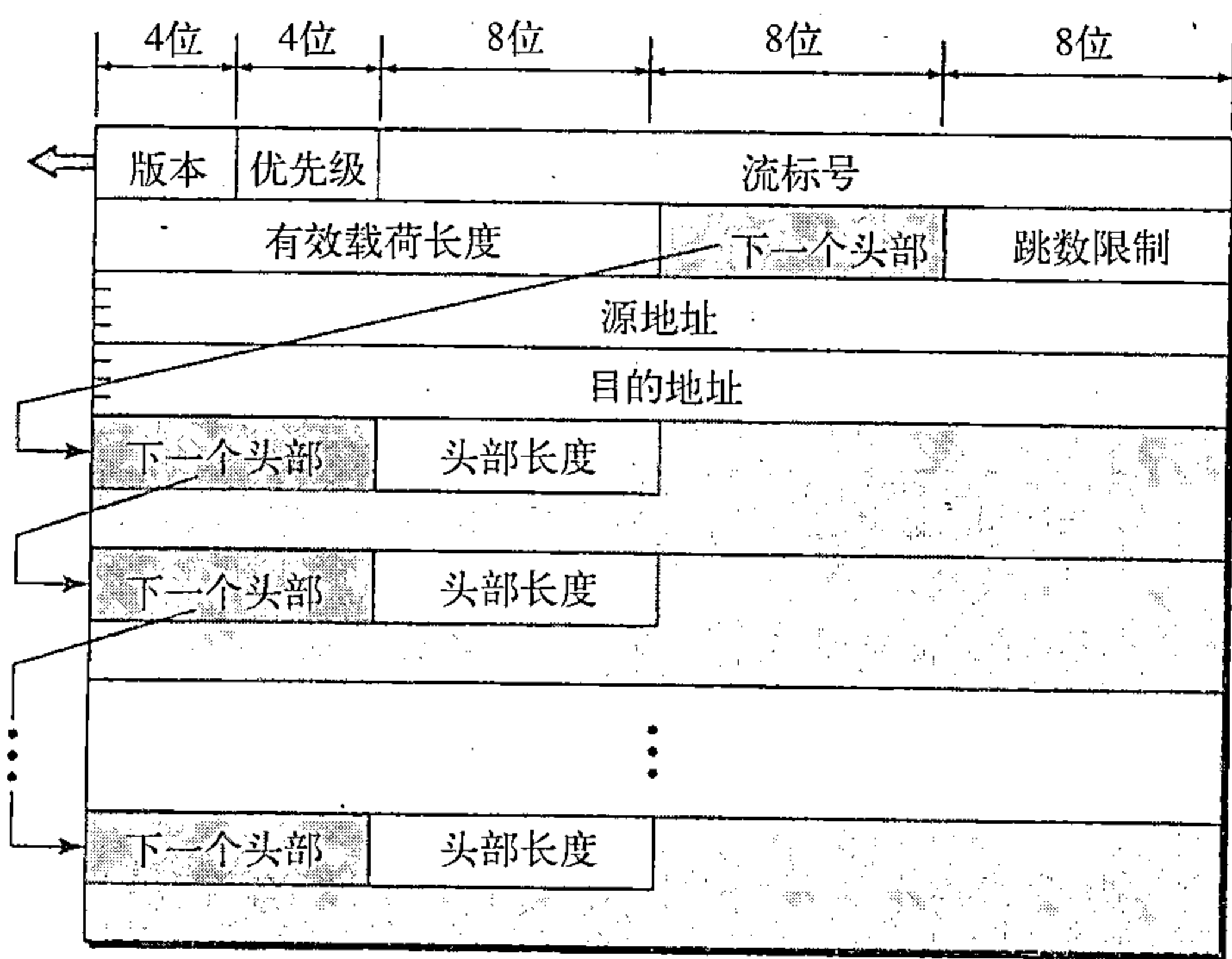


图20.16 IPv6数据报的格式

- 流标号。流标号 (flow lab) 是一个3字节 (24位) 的字段，它用来对特殊的数据流提供专门处理。我们将在后面讨论这个字段。
- 有效载荷长度。这2字节的有效载荷长度字段定义了包括基本头部在内的IP数据报的总长度。
- 下一个头部。下一个头部是一个8位的字段，它定义了数据报中跟随在基本头部之后的头部。下一个头部或者是IP所使用的可选的扩展头部，或者是上层协议的 (如UDP或TCP) 头部。每一个扩展头部也包含这个字段。表20.6给出了下一个头部的值。注意：在版本4中这个字段称为协议。
- 跳数限制。这个8位的跳数限制 (hop limit) 字段与IPv4中的TTL (生存时间) 字段所起的作用是一样的。
- 源地址。该字段是一个16字节 (128位) 的因特网地址，它是用来识别数据报的原始源端。
- 目的地址。该字段是一个16字节 (128位) 的因特网地址，通常用来识别数据报的最终目的地。然而，如果使用了源路由选择，那么该字段就包含了下一个路由器的地址。

表20.6 IPv6下一个头部的代码

代 码	下一个头部	代 码	下一个头部
0	逐跳选项	44	分段
2	ICMP	50	加密的安全有效载荷
6	TCP	51	鉴别
17	UDP	59	空 (没有下一个头部)
43	源路由选择	60	目的端选项

优先级

IPv6分组的优先级字段定义从相同源端发出的每一个分组相对于其他分组的优先级。例如，如果由于拥塞原因两个连续的数据报中必须丢弃一个，则具有较低的分组优先级 (packet

priority) 的数据报将被丢弃。IPv6将通信量划分为两大类：可进行拥塞控制的和不可进行拥塞控制的。

可进行拥塞控制的通信量 如果源端在出现拥塞时能够适应这种情况使其通信量下降，则此通信量称为可进行拥塞控制的通信量 (congestion-controlled traffic)。例如，使用滑动窗口协议的TCP协议能够很容易地对通信量进行响应。众所周知，在可进行拥塞控制的通信量中，分组可以延迟到达，或丢失，或不按序接收。可进行拥塞控制的数据被指定为从0到7的优先级，如表20.7所示。优先级0是最低的；优先级7是最高的。

表20.7 可进行拥塞控制的通信量的优先级

优 先 级	意 义	优 先 级	意 义
0	未指明的通信量	4	关注的批量数据通信量
1	后台数据	5	保留
2	不关注的数据通信量	6	交互式通信量
3	保留	7	控制通信量

优先级描述如下；

- 未指明的通信量。当不需要定义优先级时，就将优先级0分配给分组。
- 后台数据。这个组（优先级1）定义的数据通常是在后台传递，新闻的传递是一个很好的例子。
- 不关注的数据通信量。如果用户并不等待（关注）要接收的数据，则分组将被指定优先级2，电子邮件就属于这类。一个用户将电子邮件发送给另一个用户，但接收者并不知道电子邮件马上到达。此外，电子邮件通常在转发之前要经过存储，有少量延迟并不要紧。
- 关注的批量数据通信量。当传送批量数据而用户正在等待（关注）接收这个数据（可能会有些延迟时），则这种传送数据协议就使用优先级4。FTP和HTTP属于这组。
- 交互式通信量。在这一组中，需要与用户交互的协议，如TELNET被指定为第二高的优先级（6）。
- 控制通信量。控制通信量的优先级最高（7）。路由选择协议（如OSPF和RIP）以及管理协议（如SNMP）都使用这个优先级。

不可进行拥塞控制的通信量 这指的是期望最小延迟的通信量类型，丢弃分组是不希望出现的，在大多数情况下重传也是不可能的。换言之，源端不能使自己适应拥塞，实时音频和视频是这类通信量很好的例子。

优先级号8到15被指定给不可进行拥塞控制的通信量 (noncongestion-controlled traffic)。虽然还没有特殊的标准指定给这类数据，但优先级的指定是基于当丢弃一些分组时接收到的数据的质量会有多大的影响来确定的。包含较小的冗余度的数据（如低保真音频和和视频）给予较高的优先级（15），包含较小冗余度的数据（如高保真音频或视频）可以给予较低的优先级（8），见表20.8。

表20.8 不可进行拥塞控制通信量的优先级

优 先 级	意 义
8	具有最大冗余度的数据
...	
15	具有最小冗余度的数据

流标号

从特定源端向特定目的端发送的分组序列，如果需要路由器的特殊处理，则称为分组流。源地址与流标号的值组合唯一地定义了一个分组流。

对路由器来说，一个流是共享某些特性的分组序列。例如，经过相同路径，使用相同的资源，具有相同安全性，等等。支持流标号处理的路由器有一个流标号表，这个表为每一个活动的流标号设置一个项目，每一个项目定义相应的流标号所需的服务。当路由器收到一个分组时，它就从其流标号表中找出在分组中定义的流标号值所对应的项目。但注意：流标号本身并不给流标号表项目提供信息，信息是由其他方法提供的，如逐跳选项或其他协议。

在最简单形式中，流标号可用来加速路由器对分组的处理。当路由器收到一个分组时，它不用查找路由表并用路由选择算法确定下一跳的地址，而是可以很容易地在流标号表中找到下一跳的地址。

在更复杂的形式中，流标号可用来支持实时音频和视频的传输。特别是数字形式的实时音频或视频，需要高带宽、大缓存、长处理时间等资源。进程可以事先对这些资源进行预留，以保证实时数据不会因资源不够而被延迟。使用实时数据和预留这些资源需要IPv6外的一些其他协议，如实时协议（RTP）和资源预留协议（RSVP）。

为了有效地使用流标号，已定义了三个原则：

1. 流标号由源主机指定给分组，这个标号是在1到 $2^{24}-1$ 之间的随机数。当已存在的流仍处于活跃状态时，源端一定不能给新的流重复使用一个用过的流标号；
2. 如果主机不支持流标号，它就将这个字段置为0。如果路由器支持流标号，它就简单地忽略它；
3. 所有属于同一个流的分组必须具有相同的源地址、相同的目的地址、相同的优先级和相同的选项。

IPv4头部和IPv6头部的比较

表20.9 比较了IPv4和IPv6分组的头部。

表20.9 IPv4和IPv6分组头部的比较

比 较
1. IPv6取消了头部长度的字段，因为在此版本中头部的长度是固定的
2. IPv6取消了服务类型字段，优先级和流标号字段合在一起取代服务类型字段的功能
3. IPv6取消了总长度字段，替代的是有效载荷长度字段
4. 在IPv6中的基本头部中取消了标识、标记和偏移字段，这些都包含在分段扩展头部中
5. 在IPv6中，将TTL字段称为跳数限制字段
6. 协议字段被替换为下一个头部字段
7. 头部校验和取消了，因为校验和由上层协议提供，因此这一层不需要了
8. IPv4的选项字段在IPv6中被实现为扩展头部

20.3.3 扩展头部

基本头部的长度是固定的40字节，但是要使IP数据报有更多的功能，在基本头部的后面还可增加多达6个扩展头部（extension header）。这些头部中的许多都是IPv4的选项，图20.17给出了扩展头部的6种类型。

逐跳选项

当源端需要将信息传递给数据报经过

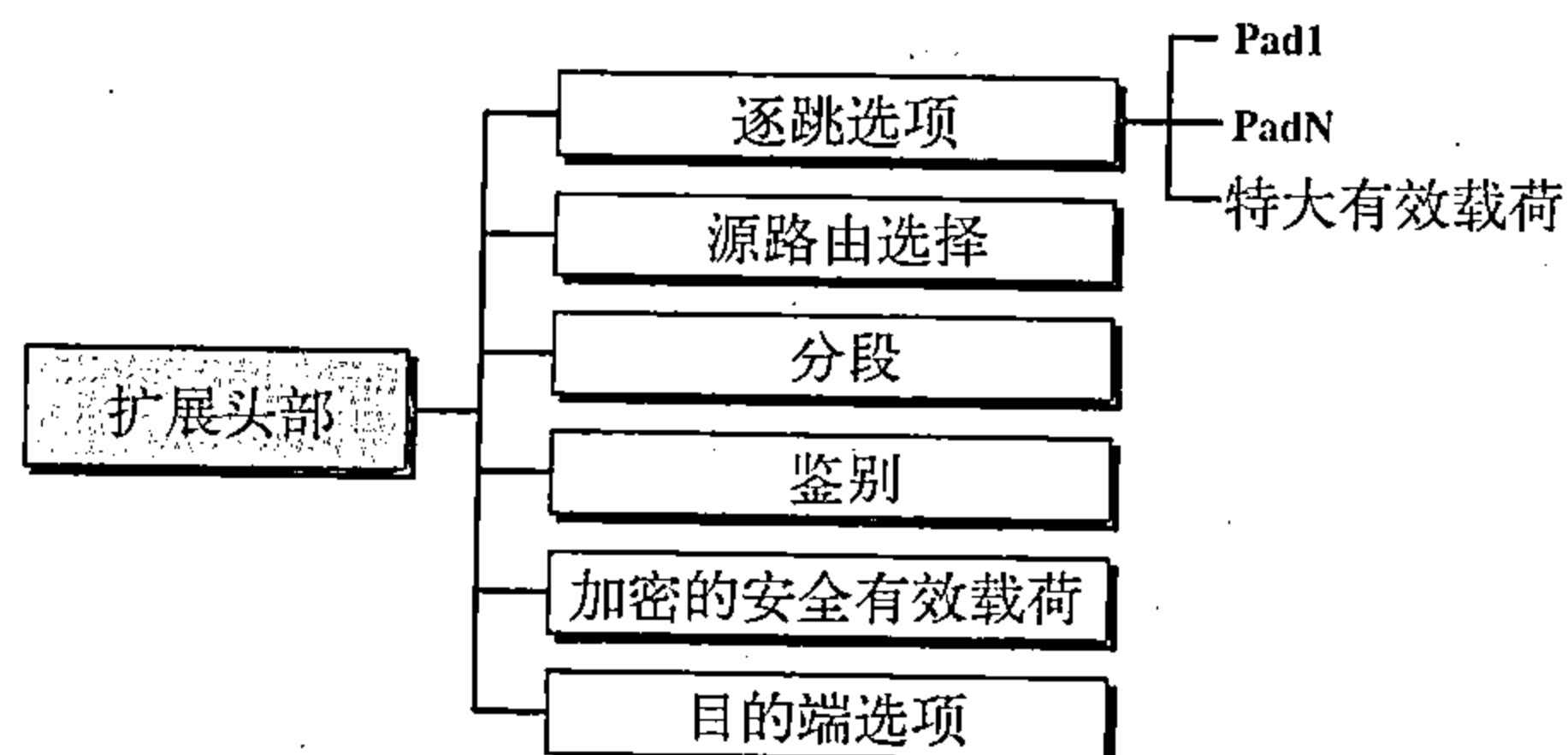


图20.17 扩展头部的类型

的所有路由器时，就需要使用逐跳选项（hop-by-hop option）。到目前为止，只定义了三种选项：Pad1、PadN和特大有效载荷（jumbo payload）。Pad1选项是1字节长，其作用是为了对齐。PadN在概念上与Pad1相似，其差别是当两个或更多字节需要对齐时，就要使用PadN。特大有效载荷选项用来定义比65 535字节更大的有效载荷。

源路由选择

源路由选择扩展头部将IPv4严格的源路由和松散的源路由的概念结合在一起。

分段

分段（fragmentation）的概念与IPv4中的一样。但分段发生的地方却不同。在IPv4中，如果数据报长度超过数据报所要经过的网络的MTU时，源端或路由器就需要将其进行分段。在IPv6中，只有原始的源端才能进行分段。源端必须使用路径MTU发现技术（path MTU discovery technique）来找出在其路径上的任何网络所支持的最小MTU，然后源端利用得到的这个知识进行分段。

鉴别

鉴别（authentication）扩展头部有双重目的：它证实报文发送者并保证数据的完整性。当我们在第31章中讨论网络安全性时，再讨论这个扩展头部。

加密的安全有效载荷

加密的安全有效载荷（encrypted security payload， ESP）是一个扩展，它提供保密性并能防止窃听。我们将在第31章中讨论这个扩展头部。

目的端选项

目的端选项（destination option）用于当源端需要将信息仅传递给目的端。中间的各路由器则不允许读取这些信息。

IPv4选项和IPv6扩展头部的比较

表20.10 比较了IPv4中的选项和IPv6中的扩展头部。

表20.10 IPv4选项和IPv6选项的比较

比 较
1. 在IPv4中的无操作和选项结束选项被替换成IPv6中的Pad1和PadN选项
2. 在IPv6中，没有记录路由选项，因为它未被使用
3. 在IPv6中，时间戳选项没有实现，因为它未被使用
4. 在IPv6中，源路由器选项称为源路由扩展头部
5. 在IPv4中的基本头部的分段字段已经移到IPv6中的分段扩展头部
6. 在IPv6中的鉴别扩展头部是新的
7. 在IPv6中，加密的安全有效载荷扩展头部是新的

20.4 IPv4到IPv6的过渡

因为因特网上的系统非常多，所以从IPv4过渡到IPv6不能突然发生。要使每一个在因特网中的系统从IPv4过渡到IPv6，需要花费相当长的时间。这种过渡必须平滑的，以防止IPv4和IPv6系统间出现任何问题。IETF已经设计了三种策略来使这一过渡时期更加平滑（见图20.18）。

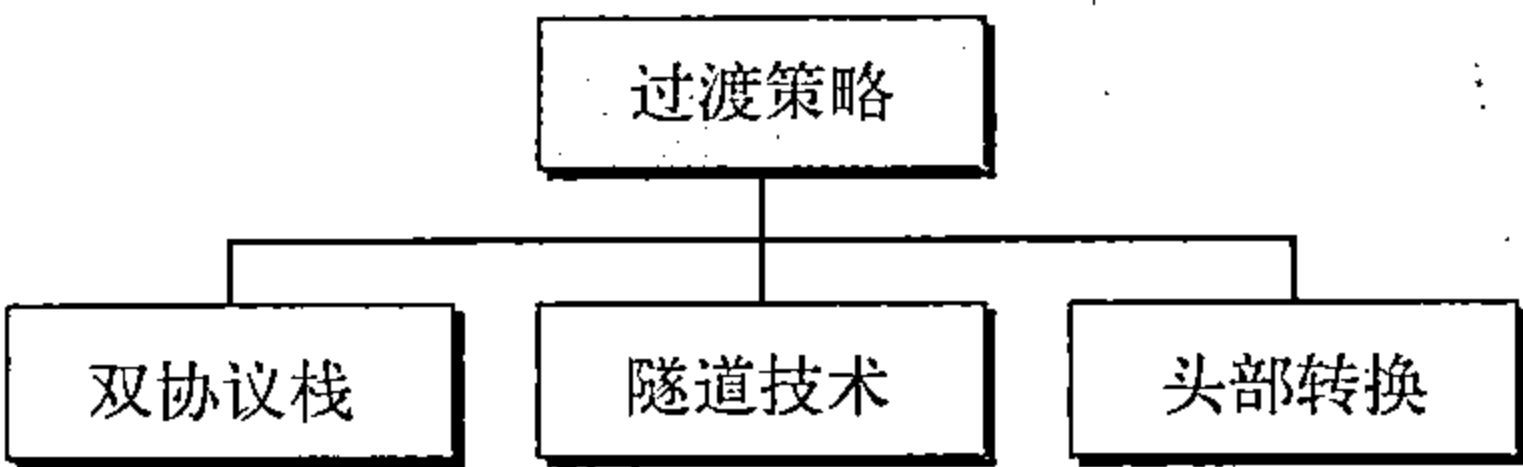


图20.18 三种过渡策略

20.4.1 双协议栈

IETF推荐所有的主机在完全过渡到第6版之前，使用一个双协议栈（dual stack）。换言之，一个站应同时运行IPv4和IPv6，直到整个因特网使用IPv6。图20.19给出了双协议栈的配置示意图。

当把分组发送到目的端时，为了确定使用哪个版本，主机要向DNS进行查询。如果DNS返回一个IPv4地址，那么源主机就发送一个IPv4分组；如果返回一个IPv6地址，就发送一个IPv6分组。

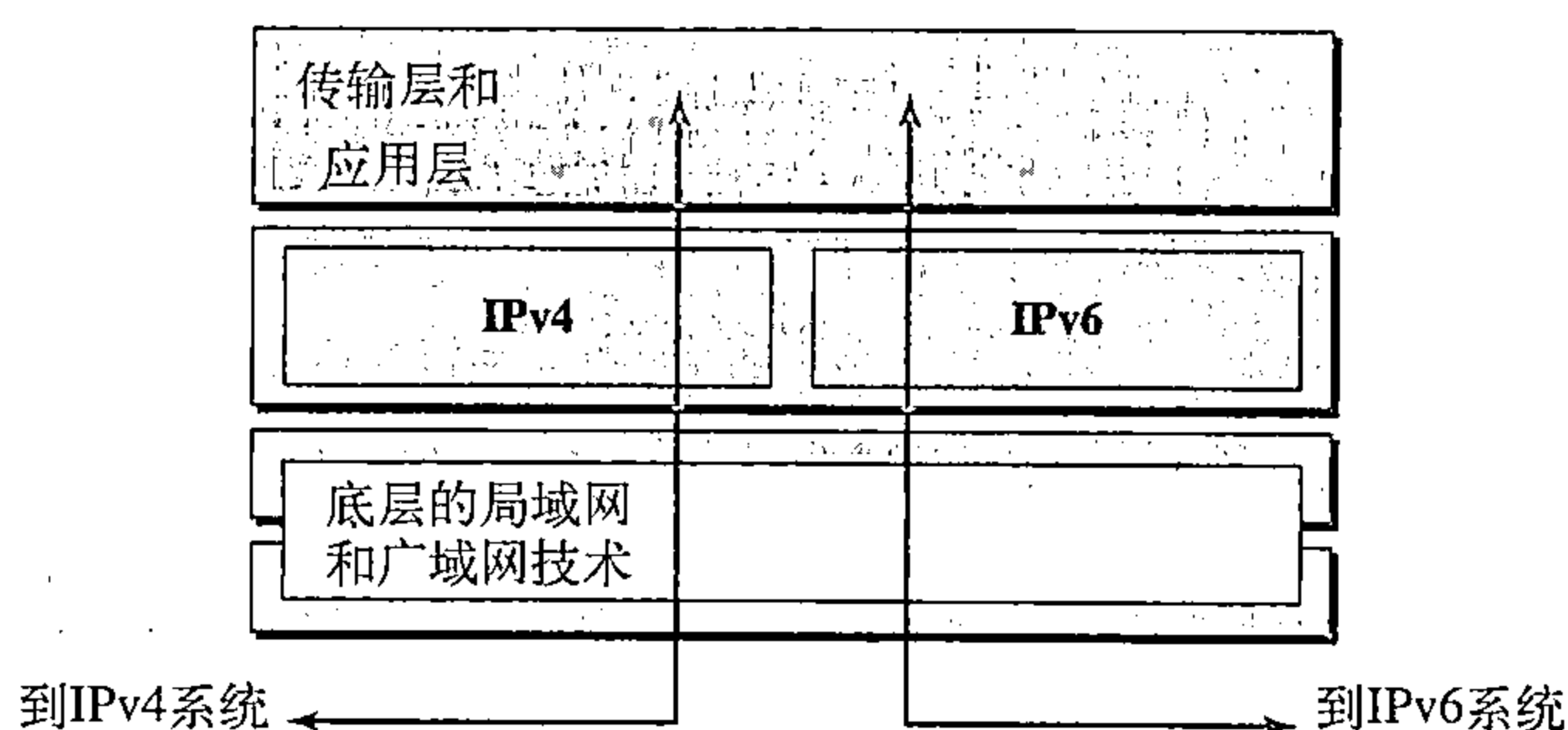


图20.19 双协议栈

20.4.2 隧道技术

当两台使用IPv6的计算机要进行相互通信，但其分组要通过使用IPv4的区域时，就要使用隧道技术（tunneling）这种策略。要经过该区域，该分组必须具有IPv4地址。因此，当进入这种区域时，IPv6分组要封装成IPv4分组，而当分组离开该区域时，再去掉这个封装。这就好像IPv6分组进入隧道一端，而在另一端流出来。为了更清楚地说明利用IPv4分组携带IPv6分组，其协议的值设置成41。隧道技术如图20.20所示。

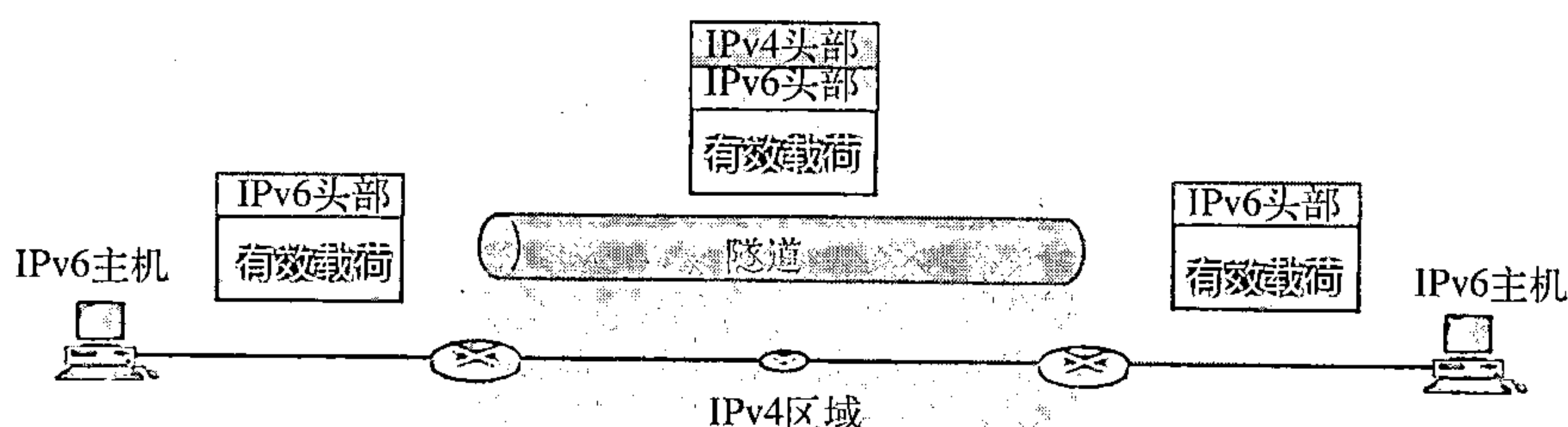


图20.20 隧道策略

20.4.3 头部转换

当因特网中绝大部分已经过渡到IPv6，但一些系统仍然使用IPv4时，需要使用头部转换（header translation）。发送方想使用IPv6，但接收方不能识别IPv6。这种情况下使用隧道技术无法工作，因为分组必须是IPv4的格式才能被接收方识别。在此情况下，头部格式必须通过头部转换彻底改变，IPv6的头部就转换成IPv4的头部（见图20.21）。

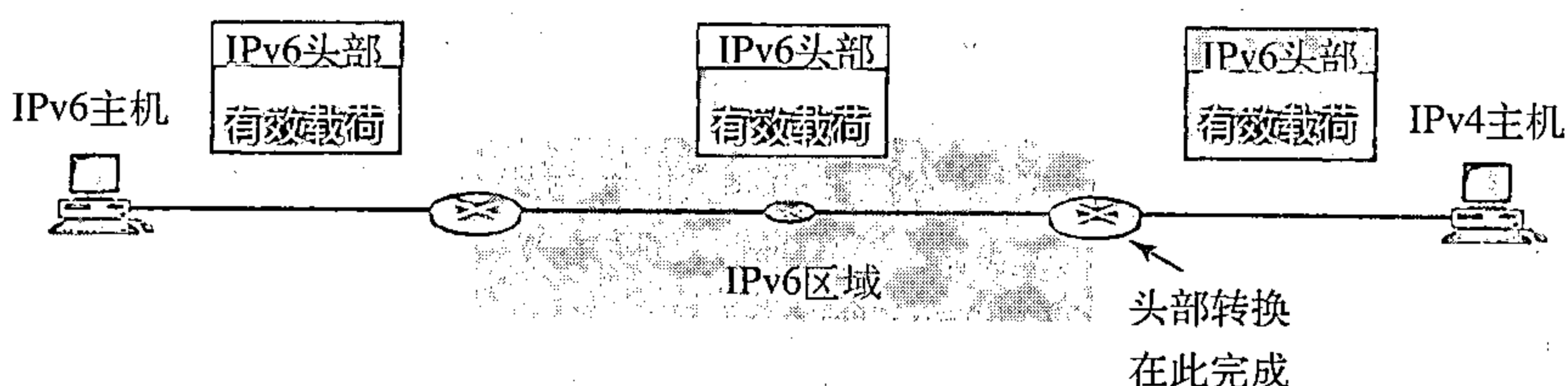


图20.21 头部转换

头部转换使用被映射的地址将IPv6地址转换成IPv4地址。表20.11列出了将IPv6分组头部转换为IPv4分组头部所使用的一些规则。

表20.11 头部转换

头部转换过程
1. 用提取最右边32位的方法，将IPv6被映射的地址改变为IPv4地址 2. 将IPv6优先级字段的值丢弃 3. 将IPv4的服务类型置为0 4. 计算IPv4的校验和，并插入到相应的字段中 5. 忽略IPv6的流标号 6. 兼容的扩展头部要转换成选项，插入到IPv4的头部中 7. 计算出IPv4头部的长度，将其插入到相应的字段中 8. 计算出IPv4分组的总长度，将其插入到相应的字段中

推荐读物

作为深入讨论与本章有关的主题，推荐下列读物和网站，在本书末列出的方括号[……]中给出参考资料。

读物

[For06]的第8章、[Ste94]的第3章、[PD03]的4.1节、[Sta04]的第18章和[Tan03]的5.6节有关于IPv4的讨论。[For06]的第27章和[Los04]有关于IPv6的讨论。

网站

- www.ietf.org/rfc.html 有关请求评论（RFC）的信息。

请求评论

IPv4的讨论可从下列请求评论找到：

760, 781, 791, 815, 1025, 1063, 1071, 1141, 1190, 1191, 1624, 2113

IPv6的讨论可以下列请求评论找到：

1365, 1550, 1678, 1680, 1682, 1683, 1686, 1688, 1726, 1752, 1826, 1883, 1884, 1886, 1887, 1955, 2080, 2373, 2452, 2463, 2465, 2466, 2472, 2492, 2545, 2590

本章小结

- IPv4是一个不可靠的无连接协议，负责源端到目的端的传递。
- 在IPv4层的分组称为数据报。一个数据报包括一个头部（20字节到60字节）和数据。数据报的最大长度是65 535字节。
- MTU是数据链路协议能够封装的最大字节数。MTU随着协议的不同而不同。
- 分段就是将一个数据报划分成若干个更小的单元，从而适应数据链路层协议的MTU。
- IPv4数据报的头部包括一个固定的20个字节的部分与一个可变的选项部分，其最大值为40个字节。
- IPv4头部中的选项部分是用于测试和调试。
- IPv4的6个选项各有其特定的功能。它们是：用于选项之间为了对齐而使用的填充符、用于选项字段结束时的填充、记录数据报所经过的路由、由发送方选择的强制性路由、选择必须经过某些路由器的路由以及记录在各路由器的处理时间。
- IPv6是网际协议的最新版本，它有128位的地址空间、修改过的头部格式、新的选项、允许扩展、支持资源分配以及增加安全措施。

- IPv6数据报由基本头部和有效载荷组成。
- 扩展头部给IPv6数据报增加了功能。
- 从IPv4过渡到IPv6所使用的三个策略：双协议栈、隧道技术和头部转换。

习题

复习题

1. 数据链路层帧传递和网络层分组传递有什么不同？
2. 无连接服务和面向连接服务有什么不同？
3. 定义分段并解释IPv4和IPv6协议为什么需要对有些分组要进行分段？在这个意义上这两个协议有没有差别？
4. 解释在IPv4协议中校验和的计算与验证。校验和计算是对IPv4分组的哪些部分进行的？为什么？如果有选项出现，是否在校验和中包括选项？
5. 解释在IPv4中需要选项，并列出本章所提到的所有选项，简要说明每个选项。
6. 比较与对照IPv4和IPv6基本头部的字段，并用一个表来说明每个字段出现或不出现。
7. IPv4和IPv6两者假定一些分组具有不同优先级，解释每个协议如何处理这个问题。
8. 比较与对照IPv4和IPv6扩展头部的选项，并做一个表来说明每个选项出现或不出现。
9. 解释在IPv6头部取消校验和的理由。
10. 列出从IPv4到IPv6过渡的三种策略。解释过渡过程中隧道策略和双协议栈策略有什么不同，每种策略应在何种情况下使用？

练习题

11. IPv4头部中的哪一个字段在经过每一个路由器时都不一样？
12. 如果总长度为1 200字节而其中的1 176字节是来自高层的数据，试计算HLEN的值。
13. 表20.5列出了不同协议的MTU，MTU的范围从296到65 535，使用大的MTU有什么好处？使用小的MTU有什么好处？
14. 已知一个被分段的数据报的分段偏移是120，如何确定第一个和最后一个字节的编号？
15. 一个分组头部长度的值能否小于5？什么时候它正好等于5？
16. 一个IPv4数据报中的HLEN值是7，试问有多少选项字节？
17. 一个IPv4数据报的选项字段长度的值是20字节，HLEN的值是多少？其二进制的值是多少？
18. 一个IPv4数据报的总长度字段的值是36，而头部长度的值是5，试问该分组携带的数据是多少字节？
19. 一个IPv4数据报携带的数据共1 024字节，如果没有选项信息，则头部长度的值是多少？
20. 一台主机发送100个数据报给其他主机，如果第一个数据报的标识号为1 024，则最后一个数据报的标识号是多少（IPv4）？
21. 一个IPv4数据报在到达时，其分段偏移为0而M位（多个分段）是0，它是一个分段吗？
22. 一个IPv4数据报在到达时，其分段偏移为100，在该分段的数据前，源端已发送了多少个分段？
23. 一个IPv4数据报在到达时，其头部有如下的信息（十六进制）：
0x45 00 00 54 00 03 58 50 20 06 00 00 7C 4E 03 02 B4 0E 0F 02
 - a. 分组是否损坏？
 - b. 有无任何选项？
 - c. 分组是否分段？
 - d. 数据的长度是多少？

- e. 分组能够经过多少路由器?
 - f. 分组的标识号是多少?
 - g. 它的服务类型是什么?
24. 一个IPv4数据报M位为0, 而HLEN是5, 总的长度值是200, 而偏移值是200。在该数据报中第一个字节和最后一个字节的编号是多少? 这是第一个分段还是最后一个分段, 或中间分段?

探究题

25. 寻找为何IPv6中有两个安全协议 (AH和ESP)。

第21章 地址映射、差错报告和多播

在第20章中，我们讨论了网络层的主要协议，网际协议（IP）。IP被设计为尽力传递协议，而没有如流控制和差错控制等一些特性，它是使用逻辑寻址的主机到主机的协议。为了使IP协议满足今天的网际互联的更多需求，还需要其他协议的帮助。

我们仍需要一些协议来创建物理地址和逻辑地址之间的映射。IP分组使用逻辑（主机到主机）地址，而这些分组需要封装成帧，帧需要物理地址（节点到节点）。我们将看到为此而设计的一个称为地址解析协议（Address Resolution Protocol, ARP）的协议，有时我们还需要逆映射（物理地址到逻辑地址）。例如，为引导一个无盘网络或给主机租用IP地址的目的，设计了三个协议：RARP、BOOTP和DHCP。

网际协议缺少流控制和差错控制，从而产生另一个协议，ICMP，它提供差错告警。报告在网络上或目的端发生拥塞和差错的类型。

IP最初是为单播传送而设计的，即一个源端和一个目的端。由于因特网对多播传送的极大需求，也就是一个源端和多个目的端。因此，IGMP提供了IP的一种多播能力。

本章将在某些方面详细讨论ARP、RARP、BOOTP、DHCP和IGMP等协议。也讨论ICMPv6，当IPv6是运行时它将也运行。ICMPv6将ARP、ICMP和IGMP合并在一个协议中。

21.1 地址映射

互联网是由一些物理网络和一些网际互联设备（如路由器）所组成。从源主机发送的分组在到达目的主机之前可能要经过许多不同的物理网络。在网络级上，主机和路由器用它们的逻辑（IP）地址进行标识。

但是，分组都要通过物理网络到达这些主机和路由器。在物理级上，主机和路由器用它们物理地址标识。物理地址（physical address）就是本地地址，它的管辖范围是本地网络。物理地址在本地范围内是唯一的，但在全局上并不必如此，所以称为物理地址，是由于物理地址通常（并非永远）是用硬件实现的。物理地址的例子是以太网协议中48位的MAC地址，它被写入在主机或路由器的网卡（NIC）上。

物理地址和逻辑地址是两种不同的标识符。这两个我们都需要，因为一个物理网络，如以太网可以在同一时间在网络层使用两个不同的协议，如IP和IPX（Novell）。同样的，在网络层的分组，如IP，也可以通过不同的物理网络，如以太网和LocalTalk（Apple）传输。

这就是说将分组传递到一台主机或路由器需要两级地址：逻辑地址和物理地址。我们需要能将一个逻辑地址映射成为它相应的物理地址，反过来的映射也需要的，这可使用静态和动态映射。

静态映射（static mapping）是创建一个表，它将一个逻辑地址与物理地址联系起来，这个表存储在网络上的每个机器上。例如，每一个机器知道其他机器的IP地址，但却不知道其物理地址，可通过查表得知物理地址。这样做有某些局限性，因为物理地址可能因以下原因而发生变化：

1. 一个机器可能会更换网卡，结果得到了一个新的物理地址；
2. 在某些局域网中，如LocalTalk，每当计算机加电时，其物理地址都要改变一次；

3. 移动的计算机可以从一个物理网络转移到另一个物理网络，这就引起物理地址的改变。要完成这些变化，静态映射必须周期性地改变，这给网络增加了非常大的开销。

在动态映射（dynamic mapping）中，每当一个机器知道两个地址（逻辑地址和物理地址）中的一个时，就可使用协议将另一个求出。

21.1.1 逻辑地址到物理地址的映射：ARP

任何时候，当主机或路由器有数据报要发送到另一个主机或路由器时，它必须有接收方的逻辑（IP）地址。如果发送方是主机，它可从DNS求得逻辑（IP）地址（见第25章）；如果发送方是路由器，它可从路由选择表求得（见第22章）。但是，IP数据报必须封装成帧才能通过物理网络。这就是说，发送方必须有接收方的物理地址。主机或路由器发送一个ARP查询分组，该分组包括发送方的物理地址和IP地址以及接收方的IP地址。因为发送方不知道接收方的物理地址，查询就在网络上广播（见图21.1）。

网络上的每一个主机或路由器都接收和处理这个ARP查询分组，但只有预期的接收者才能识别它的IP地址，并发回ARP响应分组。这个分组使用接收到的查询分组中的物理地址直接用单播发送给查询者。

在图21.1a中，左边系统（A）有一个分组要传递给IP地址为141.23.56.23的另一个系统（B）。系统A需要将分组传送给它的数据链路层进行实际的传递，但它不知道接收方的物理地址。它使用ARP的服务，请求ARP协议发送一个广播ARP请求分组，以查询IP地址为141.23.56.23的系统的物理地址。

在该物理网络上每一个系统都接收到此分组，但只有系统B才回答，如图21.1b所示。系统B发送一个包含它的物理地址的ARP回答分组。现在系统A可使用接收到的物理地址来发送到该目的地的所有分组。

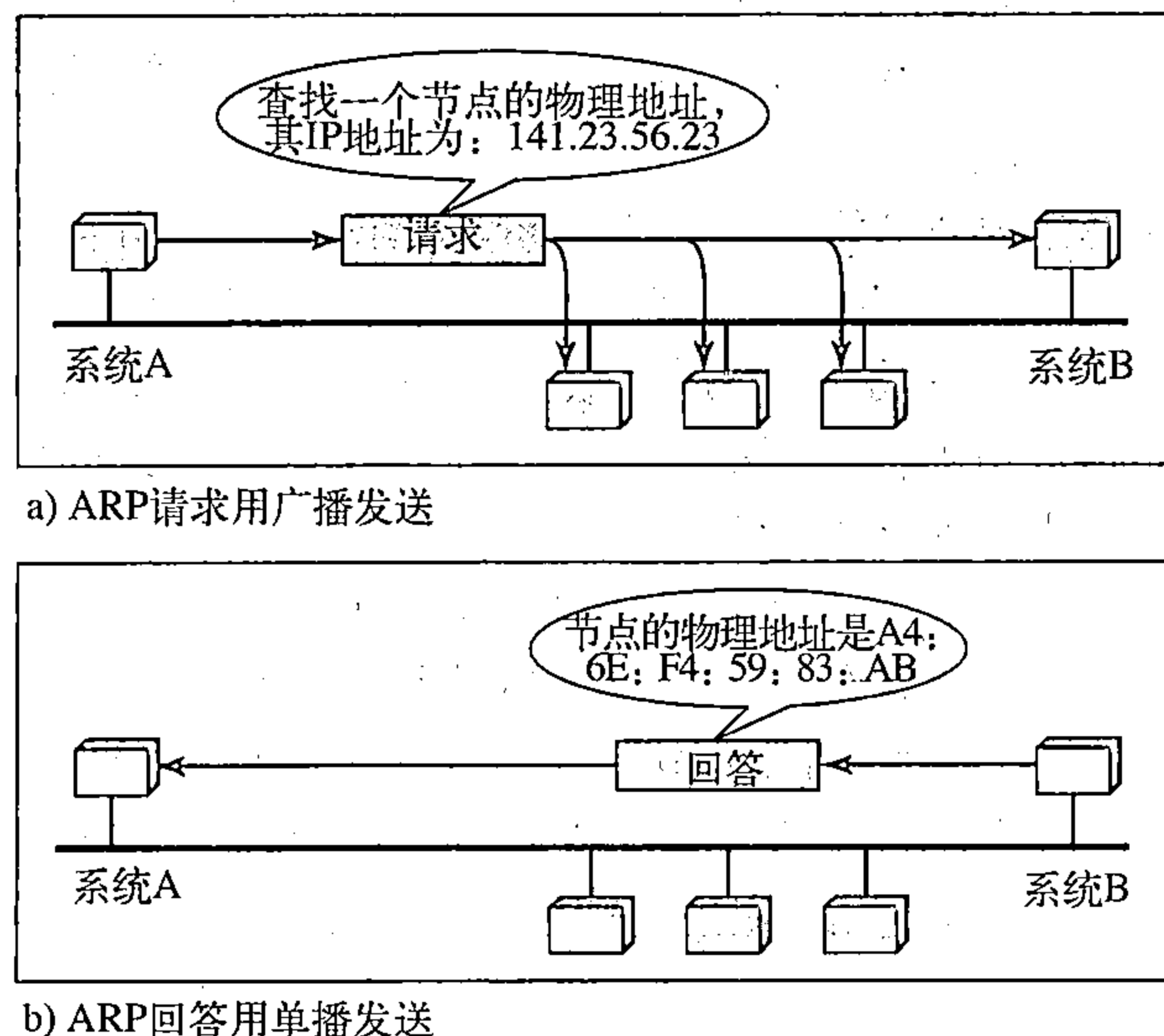


图21.1 ARP操作

高速缓存器

如果系统A对发送到系统B的每一个分组都需要广播一个ARP请求，那么对ARP协议的使用就是低效率的。它可能有对自己IP分组的广播。为了解决这一问题，ARP协议可使用高速缓存器，因为一个系统通常发送多个分组到同一目的地。接收到ARP回答的系统将它的映射

存储在高速缓存器中，保持20分钟到30分钟（除非高速缓存已满）。在以后发送ARP请求之前，系统先在它的高速缓存器中检查是否可找到它的映射。

分组格式

图21.2表示了ARP分组格式。

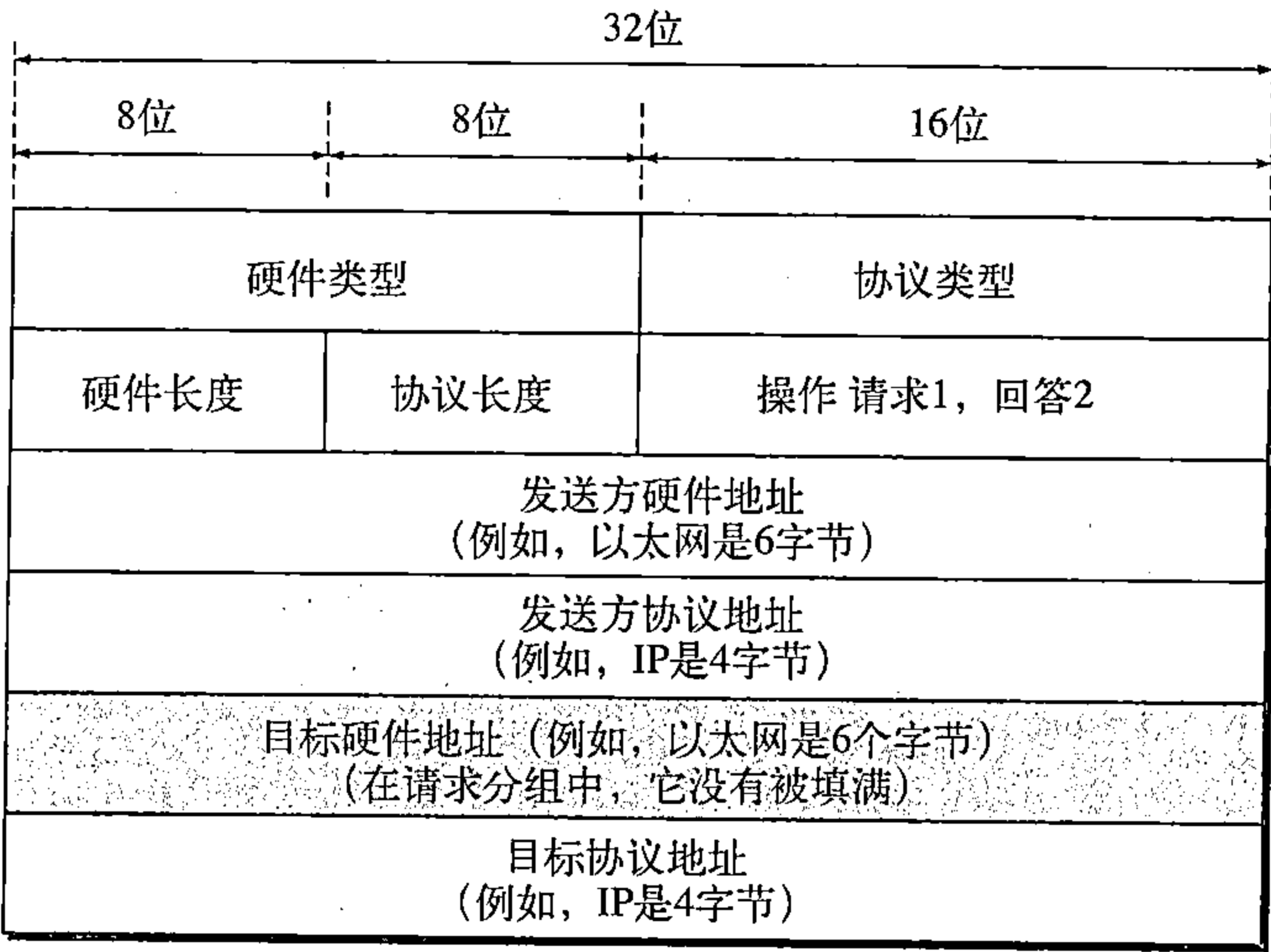


图21.2 ARP分组

ARP分组具有下列字段：

- 硬件类型。这是一个16位字段，用来定义运行ARP的网络类型。每一个局域网基于其类型被指定一个整数。例如，以太网是类型1。ARP可使用在任何物理网络上。
- 协议类型。这是一个16位字段，用来定义协议的类型。例如，对于IPv4协议，这个字段的值是0800₁₆。ARP可用于任何高层协议。
- 硬件长度。这是一个8位字段，用来定义以字节为单位的物理地址的长度。例如，对于以太网，这个值是6。
- 协议长度。这是一个8位字段，用来定义以字节为单位的逻辑地址的长度。例如，对于IPv4协议，这个值是4。
- 操作。这是一个16位字段，用来定义分组的类型。已定义了两种类型：ARP请求（1）和ARP回答（2）。
- 发送方硬件地址。这是一个可变长的字段，用来定义发送方的物理地址的长度。例如，以太网这个字段是6字节。
- 发送方协议地址。这是一个可变长的字段，用来定义发送方的逻辑（例如，IP）地址的长度。对于IP协议，这个字段是4字节。
- 目标硬件地址。这是一个可变长的字段，用来定义目标的物理地址的长度。例如，对以太网，这个字段是6字节。对ARP请求报文，这个字段是全0，因为发送方不知道目标的物理地址。
- 目标协议地址。这是一个可变长的字段，用来定义目标的逻辑地址（例如，IP地址）的长度，对于IPv4协议，这个字段是4字节。

封装

ARP分组是直接封装在数据链路帧中。例如，在图21.3中，ARP分组封装在以太网的帧中。

注意：类型字段指出了此帧所携带的数据是ARP分组。

操作

让我们讨论在一个典型的互联网上，ARP是如何工作的。首先，我们描述所包含的一些步骤，然后讨论主机或路由器需要使用ARP的四种情况。在ARP过程中，需要进行如下几个步骤：

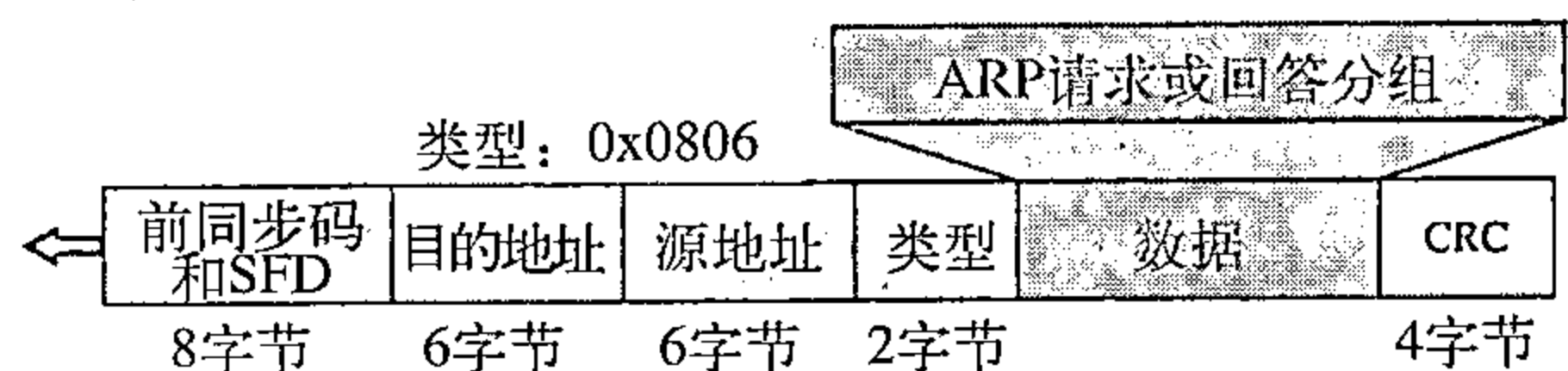


图21.3 ARP分组的封装

1. 发送方知道目标的IP地址。我们将简要地考察发送方如何得到这个地址；
2. IP请求ARP协议产生一个ARP请求报文，填入发送方的物理地址、发送方的IP地址以及目标的IP地址。目标的物理地址字段则填入0；
3. 将这个报文发送给数据链路层，在这个层它被封装成帧，使用发送方的物理地址为源地址，而将物理广播地址作为目的地址；
4. 每一个主机和路由器都接收到这个帧。因为这个帧包含了广播目的地址，所以所有的站点都将此报文送交ARP。除了目标机器外，所有的机器都丢弃该分组。目标机器识别这个IP地址；
5. 目标机器用ARP回答报文进行应答，此回答报文包含它的物理地址。报文使用单播；
6. 发送方接收到这个回答报文，它现在知道了目标机器的物理地址；
7. 携带发送给目标机器数据的IP数据报现在封装成帧，用单播发送给目的端。

四种不同的情况

下列是可使用ARP服务的四种不同的情况（见图21.4）：

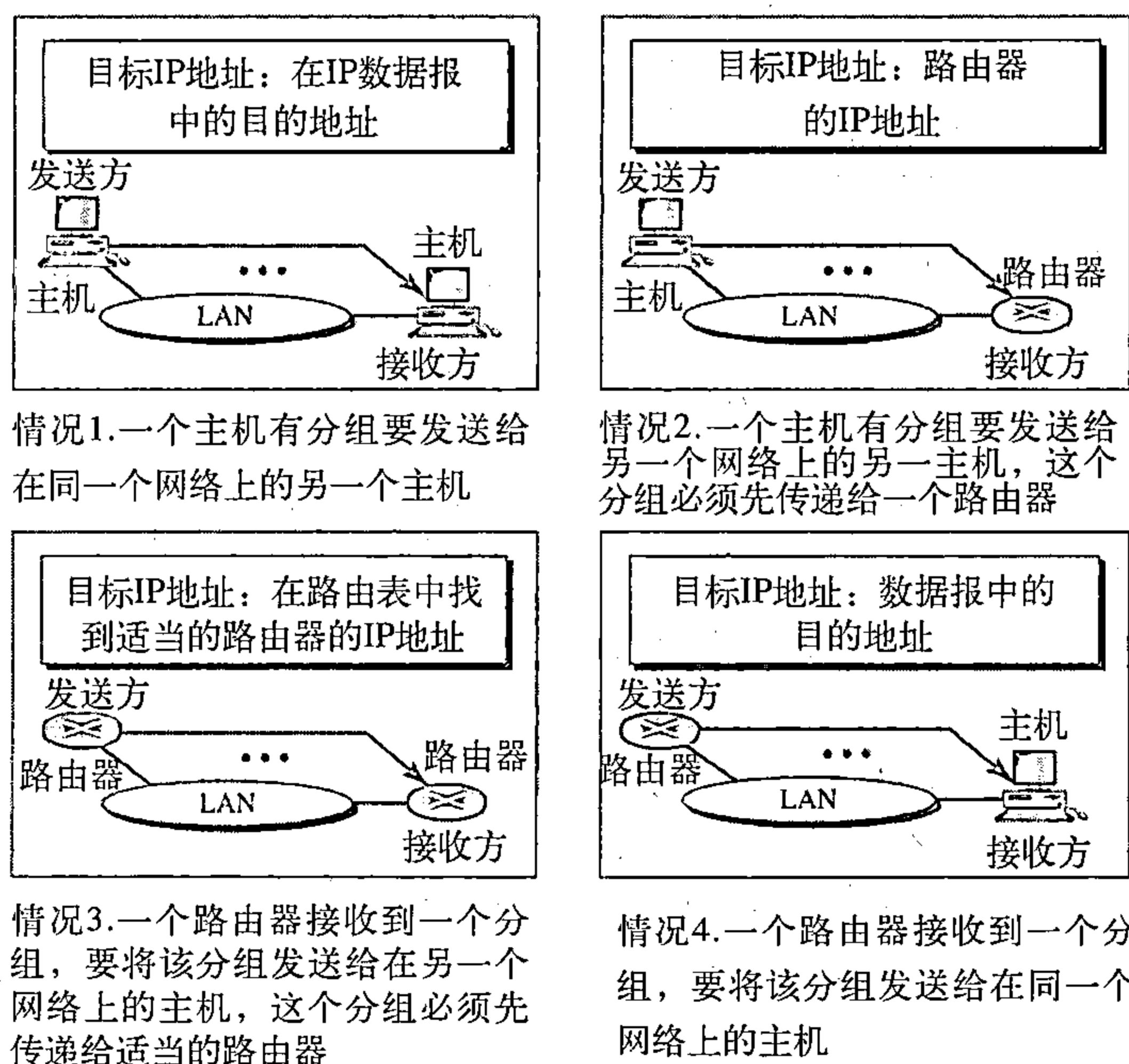


图21.4 使用ARP的四种情况

1. 发送方是一个主机，它希望将分组发送给同一个网络上的另一个主机。在这种情况下，必须将物理地址映射为逻辑地址，并将该逻辑地址作为数据报头部的目的IP地址；
2. 发送方是一个主机，它希望将分组发送给另一个网络上的另一主机。在这种情况下，

该主机查找它的路由表，找出这个目的地下一个跳（路由器）的IP地址。如果该主机没有路由表，它就要查找默认路由表的IP地址。这个路由器的IP地址就是必须映射为一个物理地址的那个逻辑地址；

3. 发送方是一个路由器，它已经收到了一个数据报，要将该数据报发送给另一个网络上的一个主机。它先检查它的路由表，找出下一个路由器的IP地址。这个下一路由器的IP地址就是必须映射为物理地址的那个逻辑地址；

4. 发送方是一个路由器，它已经收到了一个数据报，要将该数据报发送给同一网络上的一个主机。数据报的目的IP地址就是必须映射为物理地址的那个逻辑地址。

ARP请求报文是广播发送；ARP回答报文是单播发送。

例21.1

一个主机的IP地址为130.23.43.20，物理地址为B2:34:55:10:22:10，它有一个分组想要发送给另一个主机，其IP地址为130.23.43.25，物理地址为A4:6E:F4:59:83:AB（第一个主机并不知道该物理地址）。两个主机在同一个网络上。试说明ARP请求与回答分组如何封装在以太网帧中。

解：

图21.5说明了ARP请求与回答分组。注意：此时ARP数据字段是28个字节，而单个地址不适合用4字节表示界限，这就是我们为什么不以4字节界限表示这些地址。

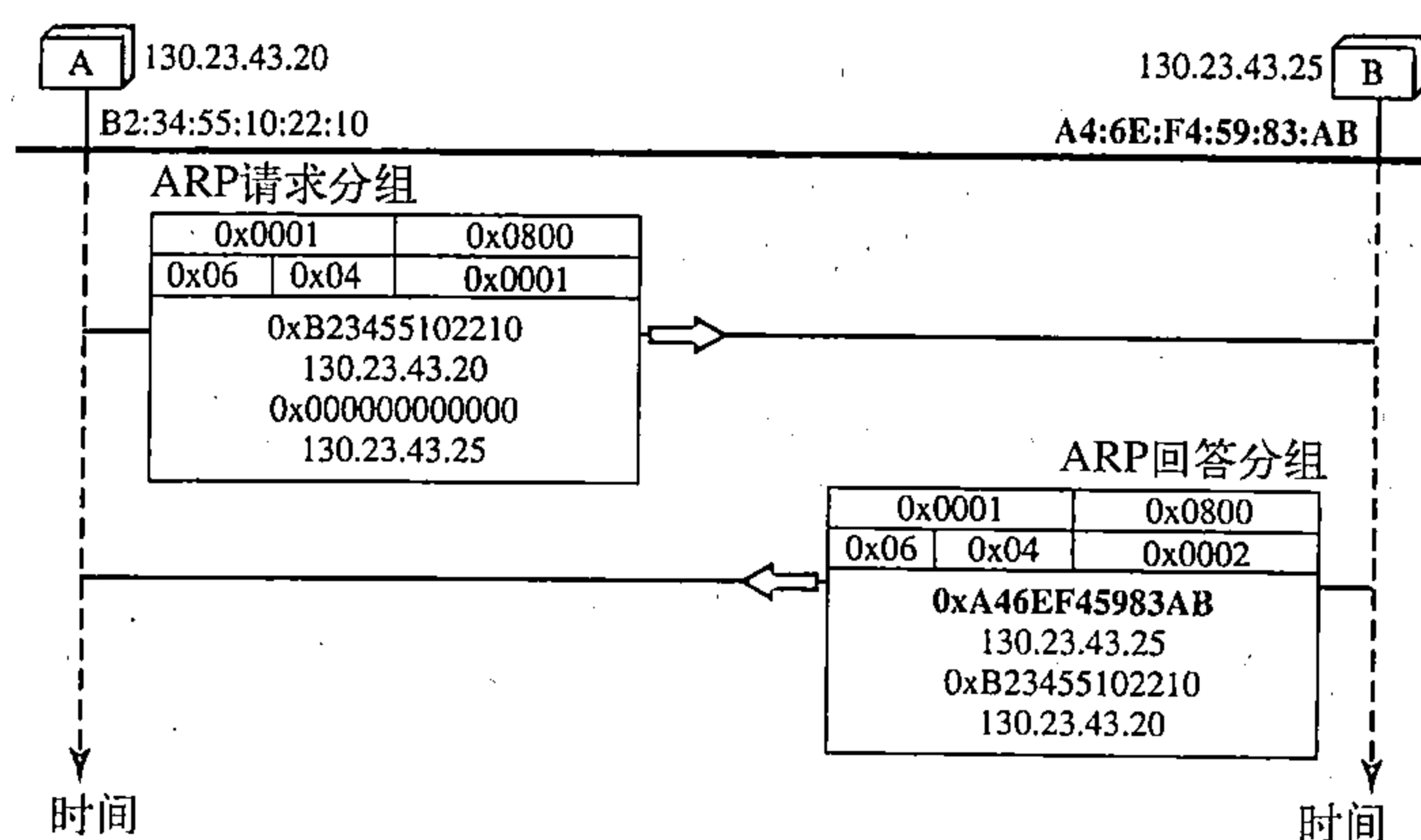


图21.5 例21.1的ARP请求与回答分组

代理ARP

有一种称为代理ARP的技术可用来产生子网化的效果。代理ARP（proxy ARP）是可以代表一组主机的ARP。每当运行代理ARP的路由器接收到一个寻找这些主机中的一个主机的IP地址的ARP请求时，路由器就发送一个ARP回答，宣布它自己的硬件（物理）地址。当这个路由器收到真正的IP分组后，它就将这个分组发送给相应的主机或路由器。

让我们给出一个实例。在图21.6

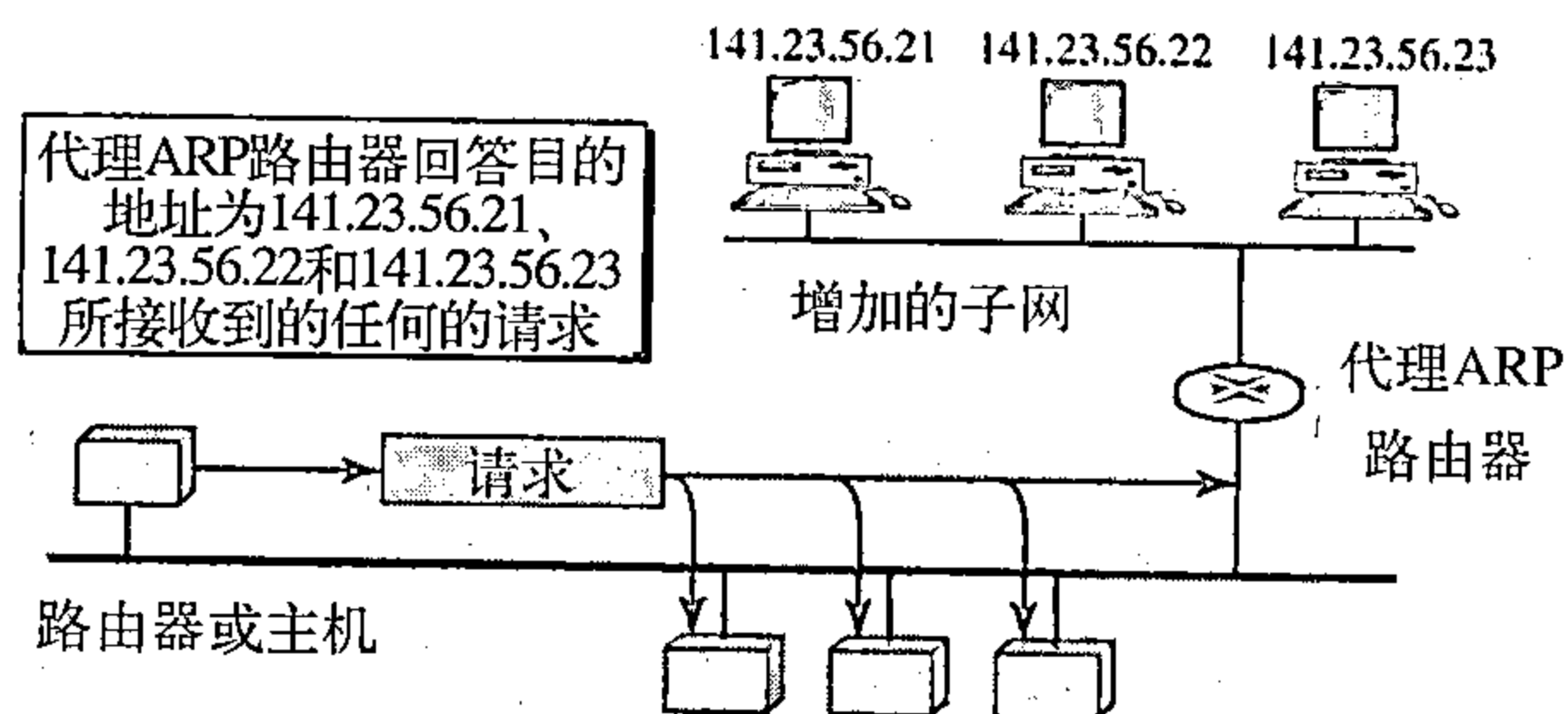


图21.6 代理ARP

中，安装在右边的主机只回答目标IP地址为141.23.56.23的ARP请求。

但是，管理员可能需要创建一个子网而不改变整个系统来识别出子网的地址。一个解决问题的方法是增加一个运行代理ARP的路由器。在这种情况下，这个路由器代表所有安装在子网上的主机。当路由器接收到一个ARP请求时，如果其目标IP地址与它所代表的主机之一的地址相匹配（141.23.56.21、141.23.56.22和141.23.56.23），则它发送一个ARP回答，并宣布其硬件地址作为目标硬件地址。当该路由器收到IP分组，它就将该分组发送给相应的主机。

21.1.2 物理地址映射到逻辑地址：RARP、BOOTP和DHCP

有两种可能的场合，当一个主机知道它的物理地址但不知道其逻辑地址：

1. 无盘站点正在被引导，站点可以通过检查其接口得到它的物理地址，但不知道它的逻辑地址；
2. 一个组织机构没有足够的IP地址分配给每一个站点，只能按需分配。站点可发送它的物理地址并请求延续一个短暂的时间。

RARP

反向地址解析协议（Reverse Address Resolution Protocol, RARP）是为仅知道物理地址的机器寻找它的逻辑地址而设计的。每一个主机或路由器都被指定一个或多个逻辑（IP）地址，这些地址是唯一的，并与机器的物理（硬件）地址无关。要创建一个IP数据报，主机或路由器就要知道它自己的IP地址。一个机器的IP地址通常可从存储在磁盘文件中的配置文件中读出。

但是，一个无盘机器通常是从ROM引导的，ROM只有少量的引导信息。ROM是由厂商安装的。它不包括IP地址，因为在网络上的IP地址是由管理员分配的。

机器可以得到其物理地址（例如，读它的NIC），这在本机是唯一的。然后就可以使用RARP协议从物理地址求得逻辑地址。先是创建一个RARP请求，并在本地网络上广播。在本地网络上知道所有IP地址的另一个机器就用RARP回答来响应。请求的机器必须运行RARP客户程序，而响应的机器必须运行RARP服务器程序。

RARP协议有一个严重的问题：在数据链路层进行广播，其广播地址在以太网中是全1，而且不能通过网络边界。这就是说，如果网络管理员有多个网络或多个子网，它需要为每个网络或子网指定一个RARP服务器。正是这个原因，使得RARP几乎不再使用，而被BOOTP和DHCP两个协议取代。

BOOTP

引导程序协议（Bootstrap Protocol, BOOTP）是一种客户机/服务器协议，设计这个协议是提供物理地址到逻辑地址的映射。BOOTP是一个应用层协议。网络管理员可以将客户机和服务器放在同一网络上或不同的网络上，如图21.7所示。BOOTP报文被封装在UDP分组中，而UDP分组本身被封装在IP分组中。

读者可能会问，当客户机既不知道它自己的IP地址（源地址）也不知道服务器的IP地址（目的地址）时，它是如何发送IP数据报的。客户机简单地使用全0作为源地址，全1作为目的地址。

BOOTP优于RARP的一点是，客户机与服务器都是应用层的进程，当在其他应用层进程中时，客户机可以在一个网络上，而服务器在相隔多个其他网络的另一个网络上。但是，必须要解决一个问题。因为客户机不知道服务器的IP地址，所以BOOTP请求是广播的。而广播的数据报不可能通过任何路由器。解决这个问题必须要有一个中间媒介，可以用一个主机

(或者配置为在应用层操作的一个路由器)作为中继。在这种情况下,该主机称为中继代理(relay agent)。中继代理知道BOOTP服务器的单播地址。当它接收到这种类型的分组时,它在单播的数据报中封装报文并向BOOTP服务器发送请求。任何一台路由器都可以路由携带单播目的地址的分组并传送到BOOTP服务器。BOOTP服务器知道来自中继代理的报文,因为在请求报文中有一个字段定义了中继代理的IP地址。在接收到回答之后,中继代理向BOOTP客户机转发它。

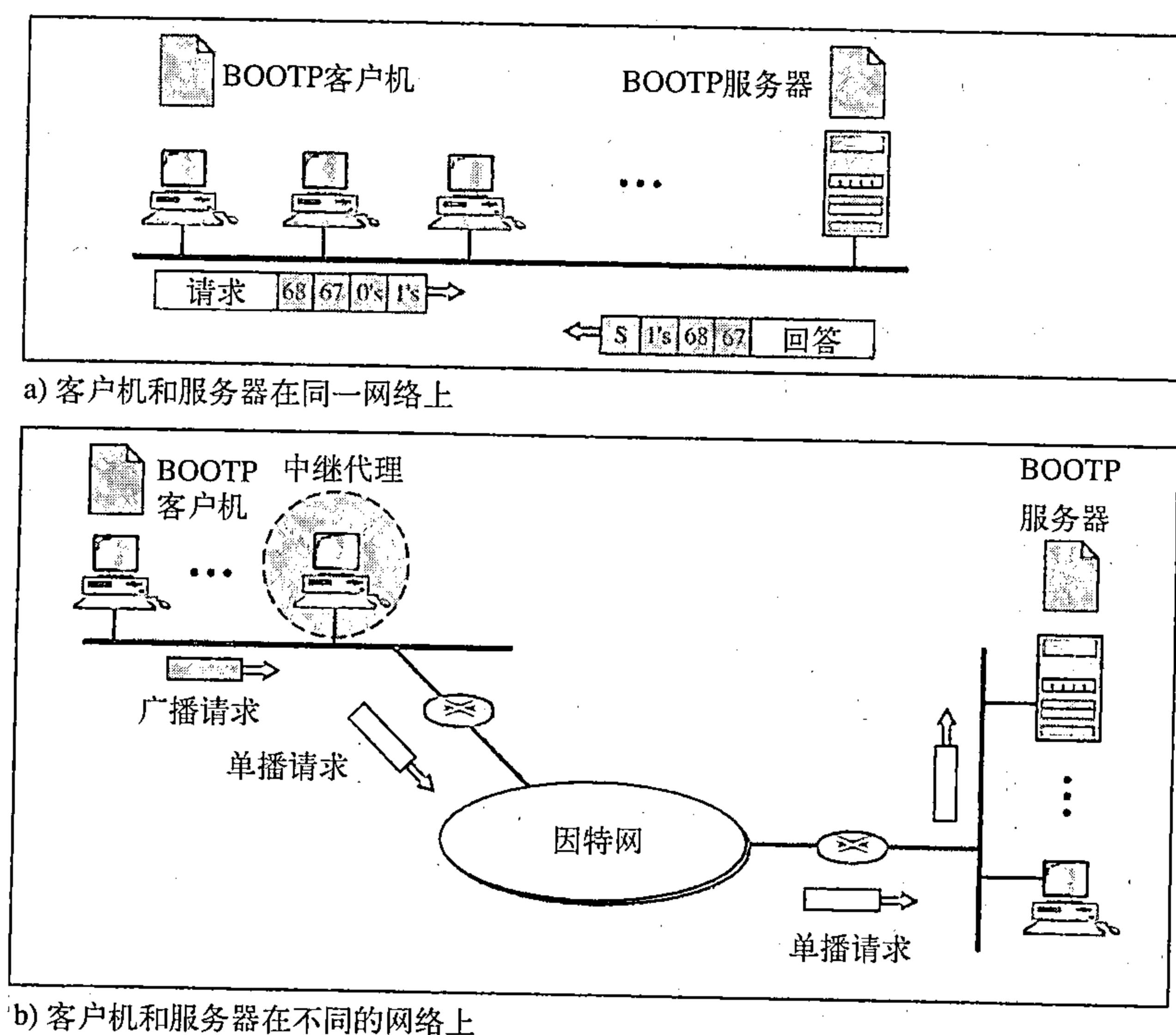


图21.7 在同一网络上和不同网络上的BOOTP客户和服务

DHCP

BOOTP不是一个动态配置协议(dynamic configuration protocol)。当客户机请求其IP地址时,BOOTP服务器查找一个表,寻找与客户机物理地址相匹配的IP地址。这意味着客户机的物理地址与IP地址的绑定必须已经存在,绑定是预先确定的。

但是,如果一个主机从一个物理网络移动到另一个物理网络时,情况又如何?如果主机想要一个临时的IP地址又如何?BOOTP不能处理这些问题,因为表中的物理地址和IP地址的绑定在网络管理员改动之前,是静态的和固定的,BOOTP是一个静态配置协议。

动态主机配置协议(Dynamic Host Configuration Protocol, DHCP)已设计出来,提供静态的和动态的地址配置,可以是人工的或自动的。

DHCP提供可以是人工的或自动的静态或动态的地址配置。

静态地址配置 这种DHCP的性能与BOOTP相同,它是与BOOTP向后兼容的,这就表示一个运行BOOTP客户机的主机可以向一个DHCP服务器请求一个静态地址。DHCP服务器有一个数据库静态地绑定物理地址和IP地址。

动态地址配置 DHCP有第二个数据库,它拥有一个可用的IP地址池。这个第二个数据库使DHCP成为动态的。当DHCP客户机请求一个临时的IP地址时,DHCP服务器就查找可用

(即未使用的) IP地址池, 然后指定一个在可协商的期间内有效的IP地址。

当DHCP客户机向DHCP服务器发送请求时, 服务器首先检查它的静态数据库, 如果数据库中存在所请求的物理地址的项目, 则返回该客户的永久IP地址。反之, 如果静态数据库中不存在该项目, 服务器就从可用的IP地址池中选择一个IP地址, 并将这个地址指定给该用户, 然后将该项目加到动态数据库中。

当主机从一个网络移到另一个网络, 或连接到一个网络后又断开连接时(如同一个用户和服务提供者的关系), DHCP是需要动态的, DHCP在有限的期间提供临时IP地址。

从可用IP地址池指定的地址是临时地址, DHCP服务器发出租用(lease)是对特定期间而言的。但租用期到时, 客户机或停止使用这个IP地址, 或更新其租用。服务器对这个更新可选择同意或不同意。如果服务器不同意, 客户机就停止使用该地址。

人工的和自动的配置 BOOTP协议的一个主要问题是人工配置IP地址到物理地址的映射表。这就是说, 每当物理或IP地址有变化时, 网络管理员必须人工地修改。而DHCP协议则既允许人工配置也允许自动配置。即静态地址可人工创建也可自动创建。

21.2 ICMP

如第20章所讨论的那样, IP提供了不可靠的和无连接的数据报传递。这样的设计是为了有效地利用资源。IP协议是一个尽力传递的服务, 它将一个数据报从它原始的源端传递到最终的目的端。但是, 它有两个缺点: 缺少差错控制和缺少辅助机制。

IP协议没有差错报告或差错纠正机制, 如果出现某些差错将会发生什么? 如果路由器因找不到一个可以到达最终目的端的路由器, 或者因生存时间字段是0而必须丢弃一个数据报时, 将会发生什么? 如果最终目的主机在预先设定的时间内不能收到所有的数据报分段, 因此将一个数据报的全部分段都丢弃时, 又会发生什么? 这是一些情况的例子, 都是出现了差错而IP协议没有内在的机制可以通知发送该数据报的主机。

IP协议还缺少一种为主机和管理查询的机制, 主机有时需要确定一个路由器或另一个主机是否是活跃的, 有时网络管理员需要从另一个主机或路由器得到信息。

因特网控制报文协议 (Internet Control Message Protocol, ICMP) 就是为了弥补上述两个缺点而设计的, 它是配合IP协议使用。

21.2.1 报文类型

ICMP报文分为两大类: **差错报告报文** (error-reporting message) 和**查询报文** (query message)。

差错报告报文向路由器或主机(目的端)报告在处理一个IP数据报时可能碰到的一些问题。

查询报文是成对出现的, 它帮助主机或网络管理员从一个路由器或另一个主机得到特定的信息。例如, 节点能够发现它们的邻站。此外, 主机能够发现和知道在它们的网络上的一些路由器的情况, 而一些路由器能帮助一个节点改变报文的路由。

21.2.2 报文格式

ICMP报文有一个8字节的头部和一个可变长的数据部分。虽然每一种报文类型的头部的格式都是不同的, 但最前面的4个字节对所有类型都是相同的, 如图21.8所示。第一个字段是ICMP的类型, 它定义报文的类型。代码字段指定了发送此特定报文类型的原因。最后一个共同的字段是校验和字段(在本章后面讨论)。头部其余部分对每种报文类型都是特定的。

在差错报文的数据部分所携带的信息可找出引起差错的原始分组。在查询报文的数据部分携带了基于查询类型的额外信息。

21.2.3 差错报告

ICMP的主要责任之一就是差错报告。虽然现代技术已经制造出很可靠的传输介质，但差错还是存在的，因而必须进行处理。如在第20章中讨论的那样，IP是不可靠的协议。这就是说，IP是不考虑差错校验和

差错控制的，ICMP就是为弥补这个缺点而设计的。然而，ICMP不能纠正差错，它只是报告差错。差错纠正留给高层协议去完成，差错报文总是发送给原始的源端，因为在数据报中关于路由唯一可用的信息就是源IP地址和目的IP地址。ICMP使用源IP地址将差错报文发送给数据报的源端（发送方）。

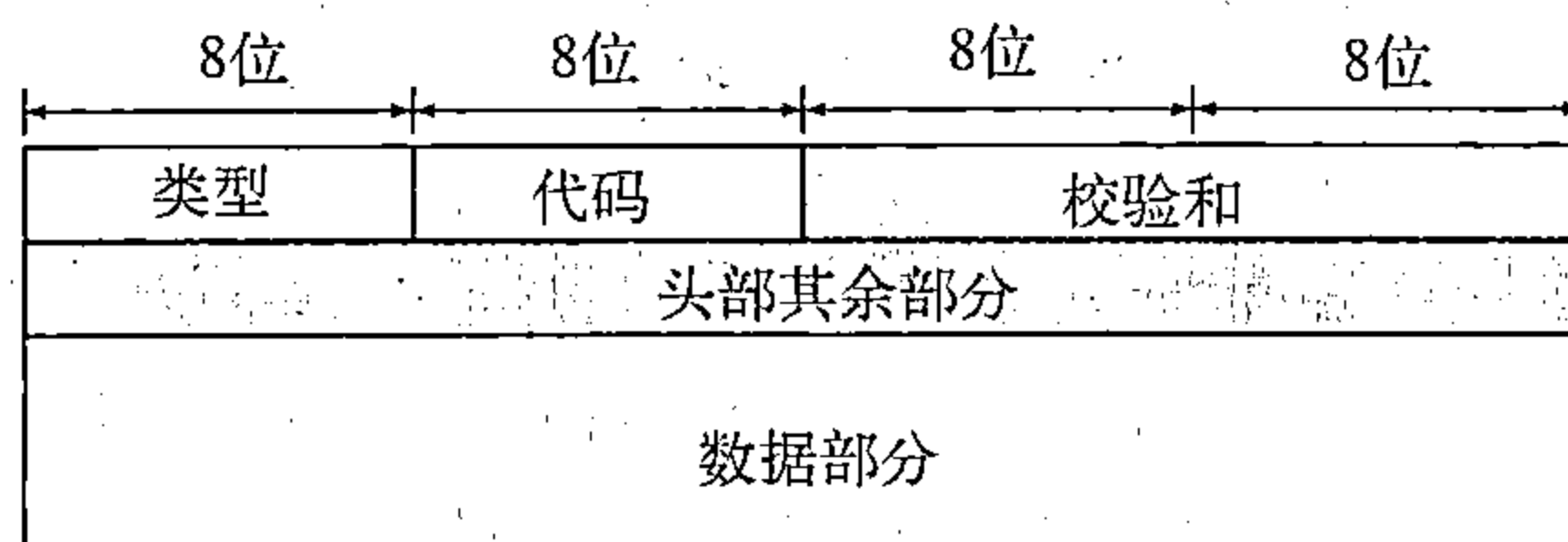


图21.8 ICMP报文一般格式

ICMP总是向原始的源方报告差错报文。

共有5种差错可处理：目的端不可达、源端抑制、时间超时、参数问题及重定向，见图21.9。

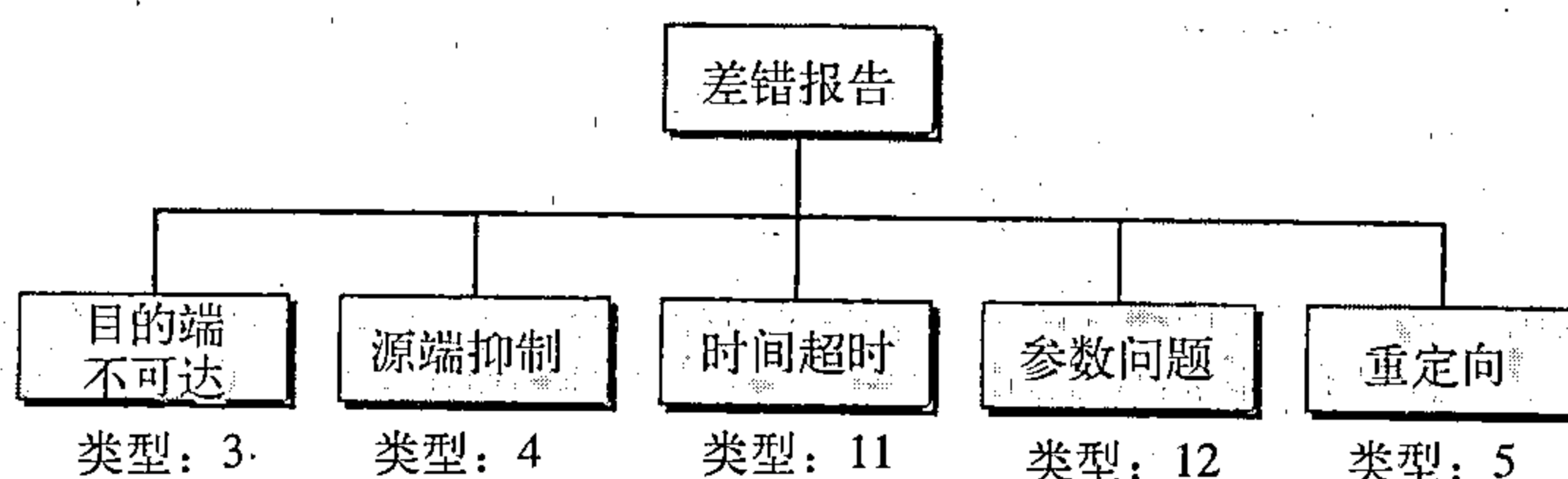


图21.9 差错报告报文

关于ICMP差错报文有下列要点：

- 对于携带ICMP差错报文的数据报，不再产生ICMP差错报文；
- 对于分段的数据报文，如果不是第一个分段则不产生ICMP差错报文；
- 对于多播地址的数据报文，不产生ICMP差错报文；
- 具有特殊地址，如127.0.0.0或0.0.0.0，不产生ICMP差错报文。

注意：所有差错报文都包括一个数据部分，而这个数据部分包括原始数据报的头部加上数据报中的前8个字节的数据。加上原始数据报中的头部就可给出原始的源端，它接收差错报文，这就是关于数据报本身的信息。要包括8个字节是因为这前8个字节提供了关于端口号（UDP和TCP）和序列号（TCP）的信息，这些我们将在第23章讨论UDP和TCP协议时看到。这个信息是需要的，这样源端可以将差错情况通知给这些协议（TCP或UDP）。ICMP形成差错分组，然后再封装成IP数据报（见图21.10）。

目的端不可达

当路由器不能够给数据报找到路由或主机不能传递数据报时，就丢弃这个数据报。然后，这个路由器或主机就发回目的端不可达报文（destination-unreachable message）给发出该数据报的源主机。注意：目的端不可达报文或者由路由器或者由目的主机创建。

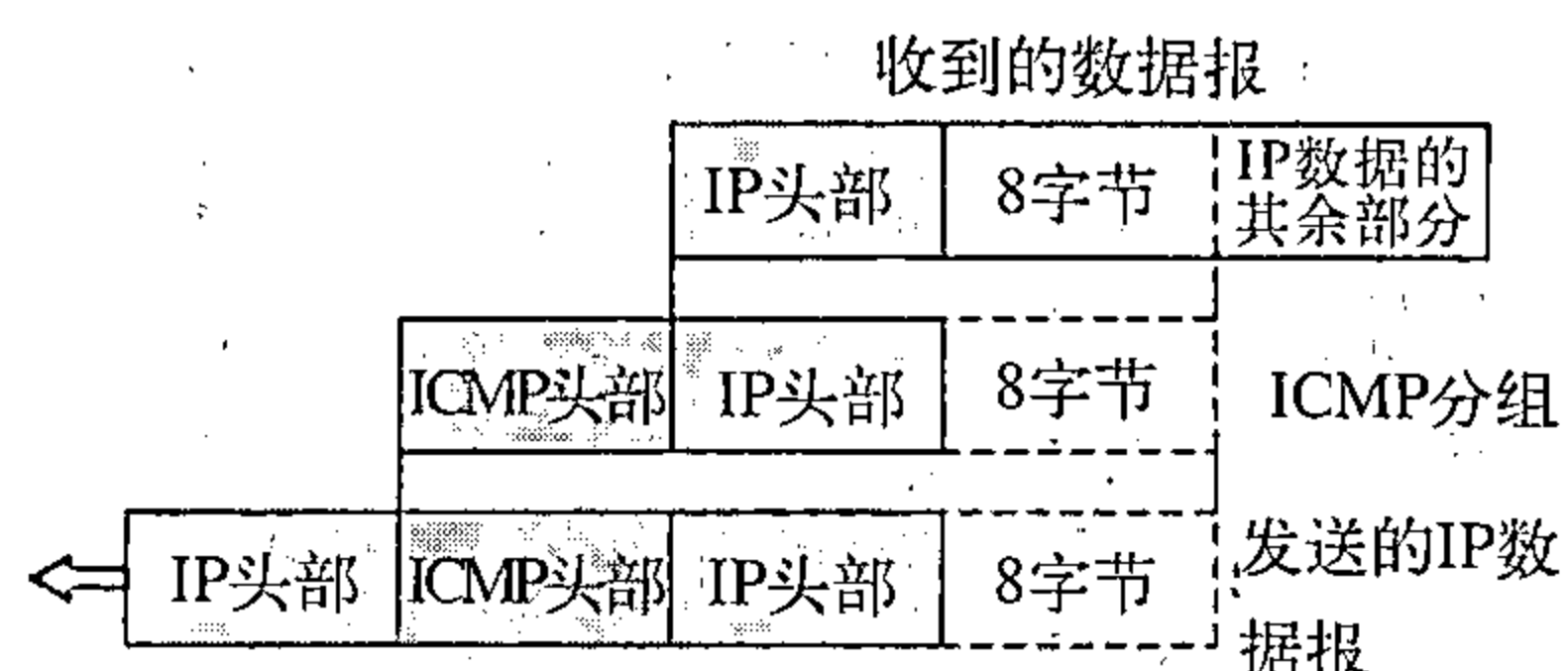


图21.10 差错报文的数据字段的内容

源端抑制

IP协议是一个无连接协议。在产生数据报的源端、转发数据报的路由器以及处理数据报的目的主机之间，并没有通信关系。这种由于缺乏通信引起的一个问题就是缺乏流量控制。IP在协议中没有嵌入流量控制机制。在运行IP时，缺乏流量控制会产生一个主要问题是拥塞。源主机从来不知道某些路由器或目的主机是否已经被过多的数据报造成超载。源主机也从来不知道它产生的数据报是否太快，以致路由器来不及转发或目的主机来不及处理。

缺乏流量控制可能会在路由器或目的主机中产生拥塞。路由器或主机中的队列长度（缓存）是有限的，这种队列是为到来的数据报等待转发（对于路由器）或等待处理（对于主机）。如果数据报的接收速率比它们被转发或处理的速率快得多，则队列就会溢出。在这个情况下，路由器或主机别无选择，只能将某些数据报丢弃。在ICMP的源端抑制报文（source-quench message）就是为了给IP增加一种流量控制而设计的。当路由器或主机因拥塞而丢弃数据报时，它就向数据报的发送方发送源端抑制报文。这个报文有两个目的。第一，它通知源端数据报已被丢弃；第二，它警告源端，在路径中的某处出现拥塞，因而源端必须放慢（抑制）发送过程。

时间超时

时间超时报文（time-exceeded message）是在二种情况下产生的：其一，如我们将在第22章中看到的，路由器使用路由表找出必须接收此分组的下一跳（下一个路由器）。如果一个或多个路由表中出现差错，那么某一个分组就可能在一个回路或循环中从一个路由器到下一个路由器，或无休止地通过一系列的路由器。正如第20章中见到的，每一数据报都有生存时间的字段控制这种情况。当数据报通过路由器时，这个字段的值减1。收到数据报的路由器在发现这个字段的值为0时，就丢弃这个数据报。但是，当丢弃这样数据报时，就要由路由器向源端发送一个时间超时报文。其二，当组成一个报文的所有分段未能在某一时限内到达主机时，也要产生时间超时报文。

参数问题

当数据报在因特网上传输时，在其头部中出现的任何二义性都可能会产生严重的问题。如果路由器或目的主机发现了这种二义性或在数据报的某个字段中缺少某个值，它就丢弃这个数据报，并向源端发送参数问题报文（parameter-problem message）。

重定向

当路由器要将分组转发到另一个网络时，它必须知道下一个适当的路由器的IP地址。如果发送方是主机，情况也是这样的。路由器和主机都必须有一个路由表，以便找出路由器或下一个路由器的地址。正如我们将在第22章看到的，路由器还要参与路由选择更新过程，并且其更新被认为是瞬时完成的，路由选择是动态的。

但是，为了提高效率，主机都不参与路由选择更新过程，因为在因特网上主机的数量比路由器要多出很多。动态地更新主机的路由表会产生不可接受的通信量。主机通常使用静态路由选择。当主机开始联网工作时，其路由表中项目的个数是很有限的。它通常只知道默认路由器这一个路由器的IP地址。因此，当主机向另一个网络发送数据报时，就将此数据报发给了这个错误的路由器。在这种情况下，收到此数据报的路由器会将该数据报转发给正确的路由器。但是，要更新主机中的路由表，它就要向主机发送重定向报文。重定向的概念如图21.11所示。主机A想要向主机B发送数据报。路由器R2显然是最有效的路由选择，但是主机A没有选择路由器R2。数据报发送到路由器R1而不是R2。R1在查找路由表后发现分组应当走R2。它把分组发送到R2，并同时向主机A发送重定向报文。这时主机A的路由表就被更新了。

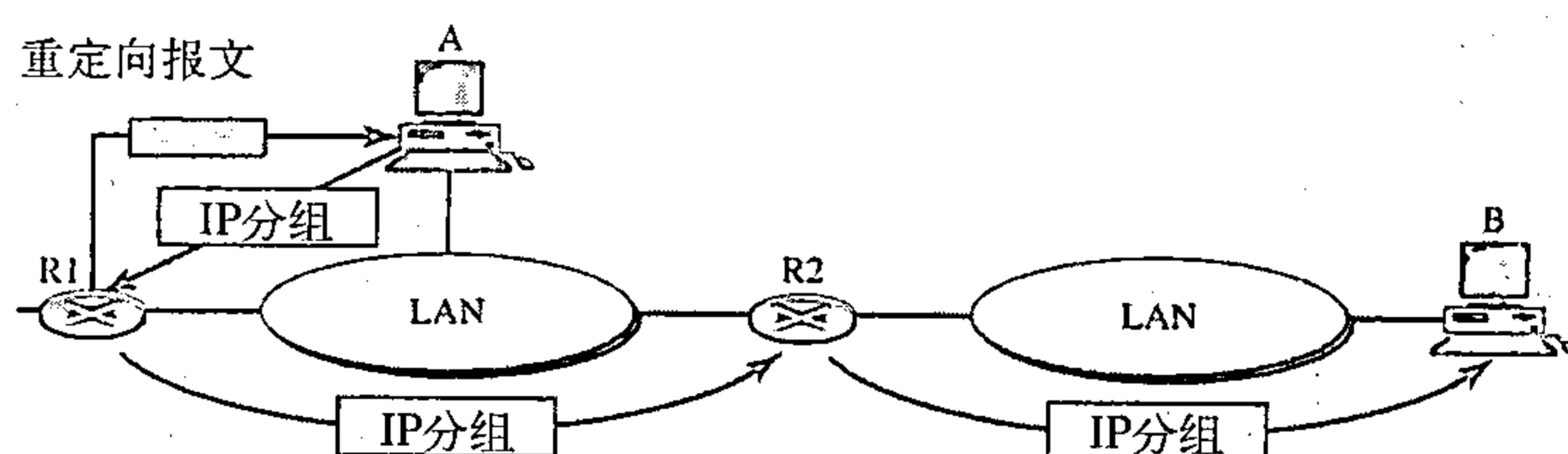


图21.11 重定向概念

21.2.4 查询

除了差错报告外，ICMP还能对某些网络问题进行诊断。这是通过使用由4对不同报文组成的查询报文来完成的，如图21.12所示。在这种类型的ICMP报文中，一个节点发送出报文，然后由目的节点用特定的格式进行回答。查询报文封装在IP分组中，然后再将IP分组封装在数据链路层的帧中。然而，在这种情况下，原始IP字节没有包含在报文中，如图21.13所示。

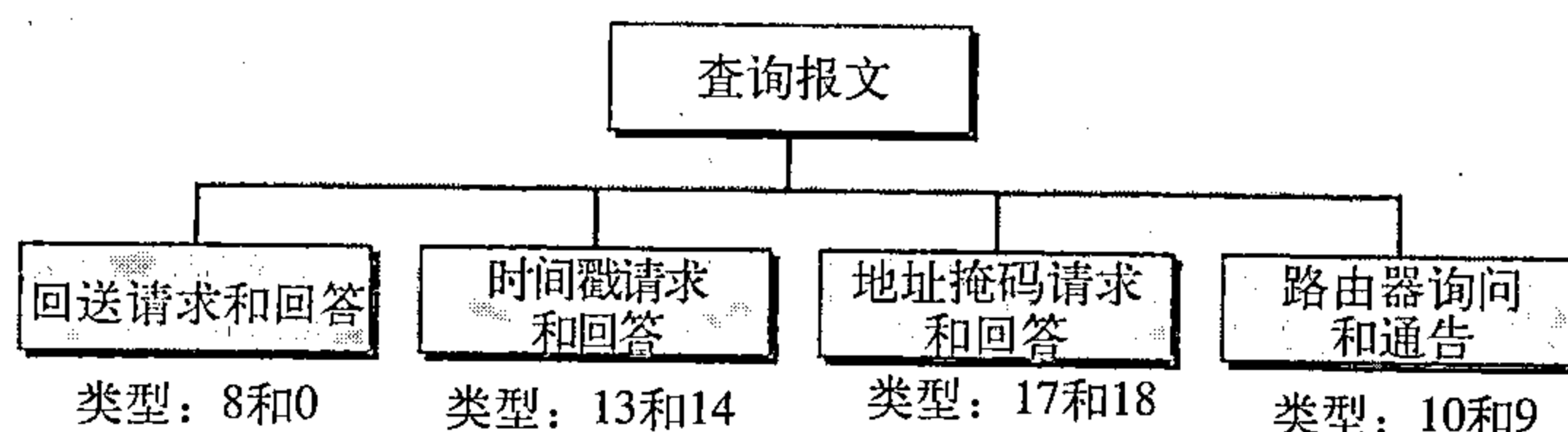


图21.12 查询报文

回送请求和回答

回送请求报文 (echo-request message) 和回送回答报文 (echo-reply message) 是为诊断目的而设计的。网络管理员和用户都可以使用这对报文发现网络问题，回送请求和回送回答组合起来确定了两个系统（主机或路由器）是否彼此能通信。

回送请求报文和回送回答报文可用来确定是否在IP级能够通信。因为ICMP报文被封装成IP数据报，发送回送请求的机器在收到回送回答报文时，就证明了在发送方和接收方之间能够使用IP数据报进行通信。此外，还证明了中间的一些路由器能够接收、处理和转发IP数据报。今天，大多数系统都提供ping命令，它可以创建一系列（不仅仅是一个）回送请求或回送回答报文，提供统计信息。我们在本章的末尾将看到这个程序的用法。

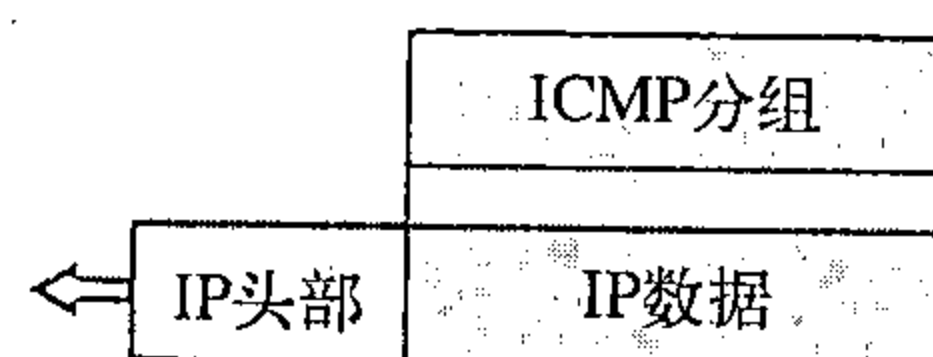


图21.13 封装ICMP查询报文

时间戳请求和回答

两个机器（主机或路由器）可使用时间戳请求和时间戳回答报文 (timestamp request and timestamp reply messages) 来确定IP数据报在两个机器之间往返所需的时间，它也可用做两个机器之间的时钟同步。

地址掩码请求和回答

主机可能知道它的IP地址，但可能不知道相应的掩码。例如，主机知道它的IP地址是159.31.17.24，但可能不知道相应的掩码/24。为了求得它的掩码，主机应向局域网上的路由器发送地址掩码请求报文 (address-mask request message)。如果主机知道该路由器的地址，它就直接向路由器发送该请求；如果它不知道地址，它就广播该报文。路由器接收到地址掩码请求报文后，用地址掩码回答报文 (address-mask reply message) 进行响应，向主机提供所需要的掩码。将这个掩码应用到完整的IP地址上，就可求得子网的掩码。

路由器询问和通告

如同我们在重定向报文的讨论中那样，主机如果想将数据发送给另一网络上的主机，就需要知道连接到它自己的网络上的路由器的地址。另外，该主机还需要知道这些路由器是否正常工作。路由器询问报文和路由器通告报文（router-solicitation message and router-advertisement message）就用于这种情况。主机可将路由器询问报文进行广播（或多播），收到询问报文的一个或多个路由器就使用路由器通告报文广播其路由选择信息。即使在没有主机询问时，路由器也可周期性地发送路由器通告报文。注意：路由器在发送通告报文时，它不仅通告自己的存在，而通告了它所知道的所有在这个网络上的路由器。

校验和

在第10章中，我们知道了校验和的概念与思想。在ICMP中，校验和的计算包括了整个报文（头部和数据）。

例21.2

图21.14表示了对一个简单的回送请求报文的校验和计算的实例。我们随机选定标识符为1，而序列号为9。将报文划分成16位（2字节）的字。将这些字相加，然后将其和取反。现在发送方就可将此值置在校验和字段。

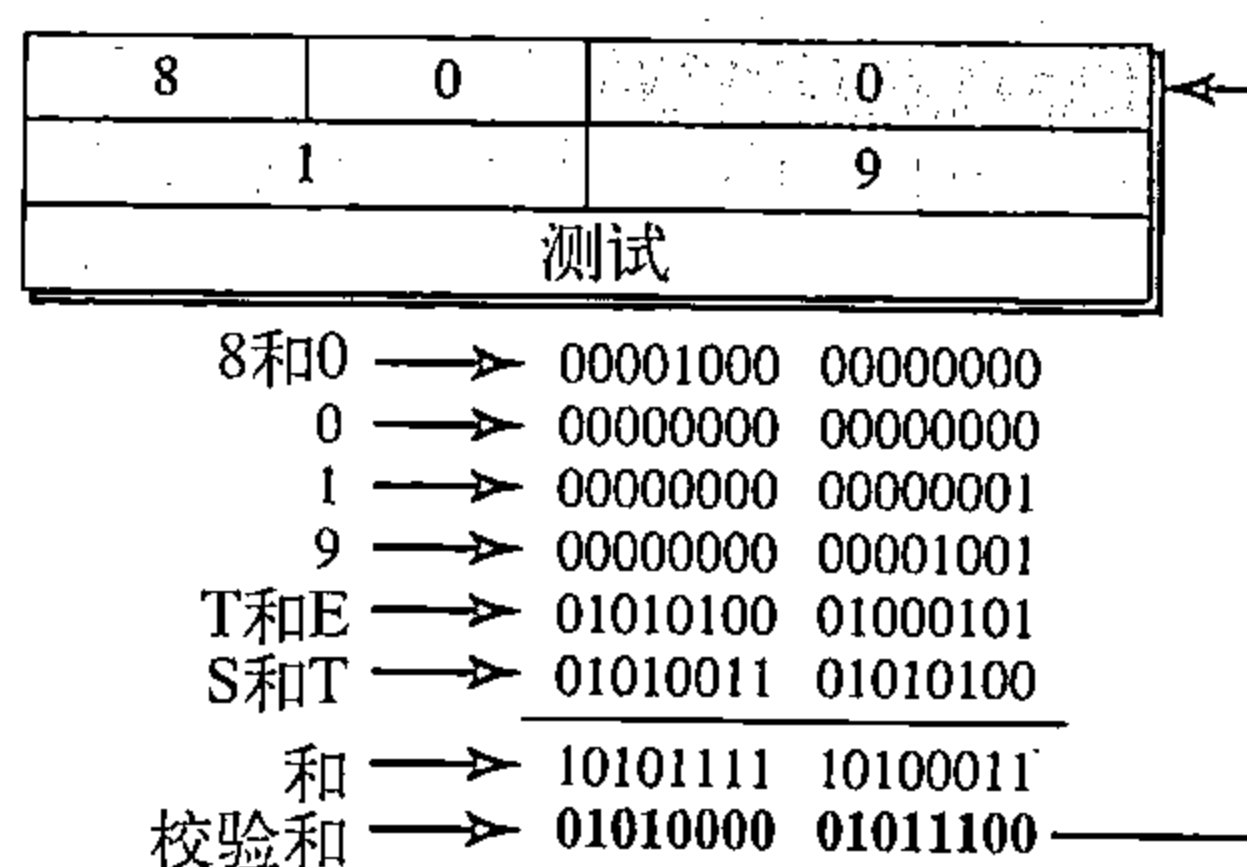


图 21.14

21.2.5 调试工具

有许多工具可用于调试因特网。我们可以确定主机或路由器是否正常工作，还可以跟踪分组的路由。我们介绍ICMP所采用两个调试工具：*ping*和*traceroute*。在对多个协议讨论之后，我们将在后面几章中介绍更多的调试工具。

Ping程序

我们可用*ping*程序来确定一个主机是否正常工作与做出响应。此处观察*ping*程序如何用于ICMP分组。

源主机发送ICMP回送请求报文（类型：8，代码：0），如果目标机器是正常工作，则它用ICMP回送回答报文响应。*ping*程序在回送请求报文和回送回答报文中设置标识符字段，并以序列号0开始；每发送一个新的报文该序列号加1。注意：*ping*程序能计算出往返时间，*ping*程序在报文数据部分中插入发送时间。往返时间（RRT）的值是收到回答报文的时间减去发送请求报文时间的值。

例21.3

我们用*ping*程序测试服务器fhda.edu，其结果如下：

```
$ ping fhda.edu
PING fhda.edu (153.18.8.1) 56 (84) bytes of data.
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=0    ttl=62    time=1.91 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=1    ttl=62    time=2.04 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=2    ttl=62    time=1.90 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=3    ttl=62    time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=4    ttl=62    time=1.93 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=5    ttl=62    time=2.00 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=6    ttl=62    time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=7    ttl=62    time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=8    ttl=62    time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=9    ttl=62    time=1.89 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=10   ttl=62    time=1.98 ms
```

---fhda.edu ping统计---

已发送11个分组，11个分组都被接收到，丢弃分组为0%，共计时间为10 103ms
往返最小时间/平均时间/最大时间=1.899/1.955/2.041ms

*ping*程序发送报文的序列号从0开始，每次探测给出这次的往返时间。封装了ICMP报文的IP数据报的TTL（生存时间）字段已被置为62，这就是说该分组的传输不能超过62跳。一开始*ping*程序定义数据部分为56个字节，IP数据报总长度为84字节。这是显然的，这是因为我们要增加ICMP头部8字节和IP头部20字节，所以其结果是84字节。但是注意：每次探测*ping*程序定义的字节个数为64，这是ICMP分组的总长度（56+8）。如果不用中断键（例如，ctrl+c）停止*ping*程序，则它一直继续下去。当程序被中断后，它打印出检查的统计结果。它告诉我们发送的分组数以及接收到的分组数、总的时间和往返最小、最大平均时间。有些系统还可以打印出更多的信息。

Traceroute程序

UNIX中的*traceroute*程序或Windows中的*tracert*程序可用于追踪一个分组从源端到目的端所经过的路由。*Traceroute*程序的应用与第20章中的IP数据报的松散的路由选项和严格路由选项相类似。本章我们将这个程序和ICMP数据报一道使用，程序巧妙地利用时间超时与目的端不可达两种ICMP报文找出分组经过的路由。这是一个UDP提供服务的应用程序（见第23章）。让我们用图21.15表示*traceroute*程序的思想。

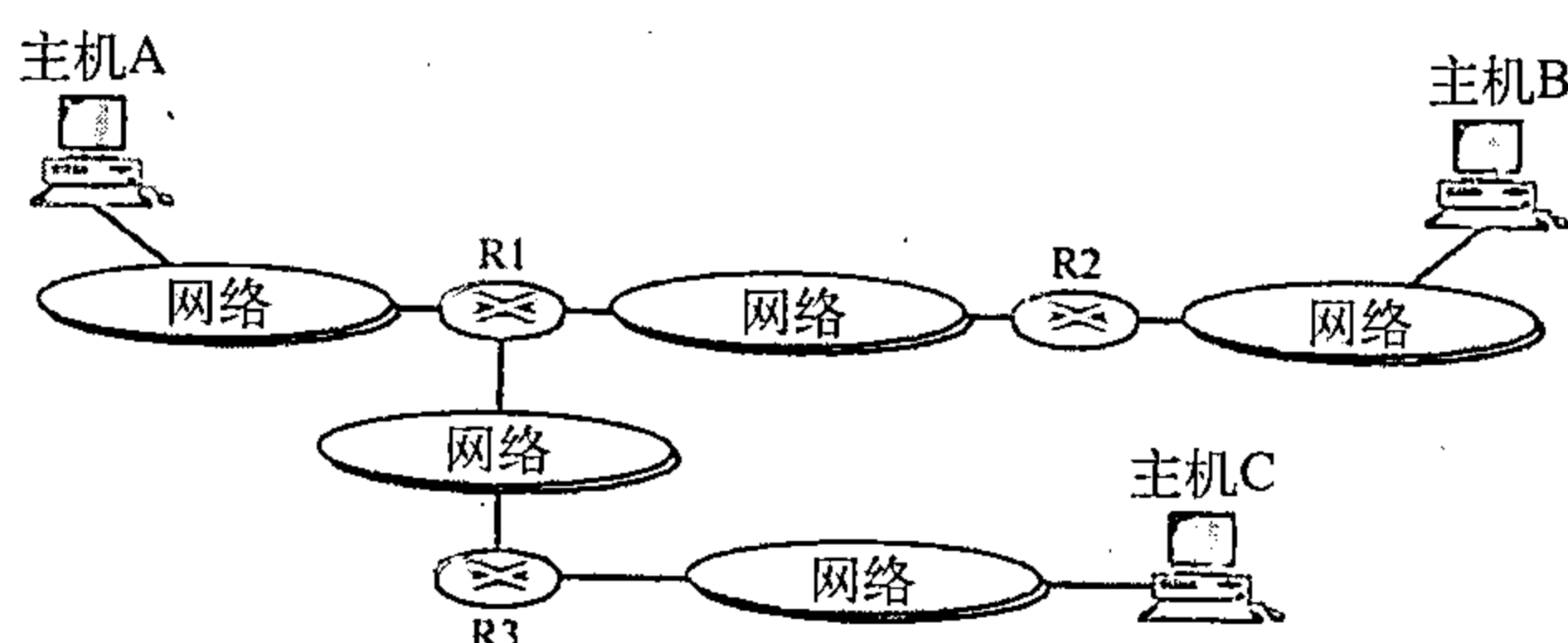


图21.15 *traceroute*程序的操作

给定图中的网络拓扑结构，我们知道主机A向主机B发送一个分组要经过路由器R1和路由器R2。但是，多数

情况下我们并不知道网络拓扑结构，主机A到主机B可能存在多条路由。*traceroute*程序利用ICMP报文和IP分组的TTL（生存时间）字段找出路由。

1. *traceroute*程序用下列步骤求得路由器R1的地址和主机A到路由器R1的往返的时间。

a. 主机A上的*traceroute*程序利用UDP协议发送一个分组到目的主机B，该报文被封装在TTL字段值为1的IP分组中。程序记住发送分组的时间。

b. 路由器R1接收到分组并将TTL的值减1成为0，然后路由器R1丢弃该分组（由于TTL字段的值为0），但是路由器R1发送一个时间超时ICMP报文（类型：11，代码：0）表示TTL的值为0而该分组已被丢弃。

c. *traceroute*程序接收到该ICMP报文，并利用封装了ICMP报文的IP分组的地址求得路由器R1的IP地址。程序也记住分组到达的时间，这个时间与a步记下的时间差就是往返时间。

*traceroute*程序重复步骤a到c三次求得一个较好的平均往返时间。第一次往返时间可能会比第二次或第三次时间长，因为第一次需用ARP程序去查找R1的物理地址，而第二次和第三次往返时，ARP在它的高速缓存中有该地址。

2. *traceroute*程序重复上面的步骤求得R2的地址以及主机A到R2的往返时间。但是，在这一步，TTL字段设置为2。因此，R1转发这个报文，而R2丢弃它并发送时间超时ICMP报文。

3. *traceroute*程序重复步骤2求得主机B的地址以及主机A到主机B往返时间。当主机B接收到分组时，TTL值减1。但主机B不丢弃该报文，因为它已到达了它的最终的目的地。ICMP报文如何发送回到主机A？*traceroute*程序在此处采用一种不寻常的策略。UDP分组的目的端口被设置UDP协议不支持的端口。当主机B接收到该分组，它找不到接收该传递的应用程序。它

丢弃该分组并发送一个ICMP目的端不可达报文（类型：3，代码：3）到主机A。注意：这种情形在路由器R1和R2不会发生，因为路由器不检查UDP的头部。*traceroute*程序记录到达的IP数据报的目的地址以及记住往返时间。接收到代码为3的目的端不可达的ICMP报文就表示已求得全部路由，此时不需要再发送分组。

例21.4

我们用*traceroute*程序求计算机voyager.deanza.edu到服务器fhda.edu的路由，下面表示其结果：

```
$ traceroute fhda.edu
traceroute to fhda.edu (153.18.8.1), 30 hops max, 38 byte packets
 1 Dcore.fhda.edu      (153.18.31.254)    0.995 ms   0.899 ms   0.878 ms
 2 Dbackup.fhda.edu    (153.18.251.4)     1.039 ms   1.064 ms   1.083 ms
 3 tiptoe.fhda.edu     (153.18.8.1)       1.797 ms   1.642 ms   1.757 ms
```

命令后无序列的行表示了目的端是153.18.8.1，TTL值是30跳，该分组包含38个字节：IP头部20个字节、UDP头部8个字节和应用数据10个字节。*traceroute*程序使用应用数据存储分组的轨迹。

第一行表示访问第一个路由器，该路由器名称是Dcore.fhda.edu，其IP地址为135.18.31.254。第一个往返时间是0.995ms，第二个往返时间是0.899ms，第三个往返时间是0.878ms。

第二行表示访问第二个路由器，该路由器名称是Dbackup.fhda.edu，其IP地址为153.18.251.4。也显示了三个往返时间。

第三行表示目的主机。我们知道这是目的主机，因为没有更多的行。目的主机是服务器fhda.edu，但它的名称是tiptoe.fhda.edu，其IP地址为153.18.8.1。也显示了三个往返时间。

例21.5

这个例我们追踪了一个较长的路由，到xerox.com的路由。

```
$ traceroute xerox.com
traceroute to xerox.com (13.1.64.93), 30 hops max, 38 byte packets
 1 Dcore.fhda.edu      (153.18.31.254)    0.622 ms   0.891 ms   0.875 ms
 2 Ddmz.fhda.edu       (153.18.251.40)     2.132 ms   2.266 ms   2.094 ms
 3 Cinic.fhda.edu      (153.18.253.126)    2.110 ms   2.145 ms   1.763 ms
 4 cenic.net           (137.164.32.140)    3.069 ms   2.875 ms   2.930 ms
 5 cenic.net           (137.164.22.31)     4.205 ms   4.870 ms   4.197 ms
 .....
14 snfc21.pbi.net      (151.164.191.49)    7.656 ms   7.129 ms   6.866 ms
15 sbcglobal.net       (151.164.243.58)     7.844 ms   7.545 ms   7.353 ms
16 pacbell.net         (209.232.138.114)    9.857 ms   9.535 ms   9.603 ms
17 209.233.48.223      (209.233.48.223)    10.634 ms  10.771 ms  10.592 ms
18 alpha.Xerox.COM     (13.1.64.93)        11.172 ms  11.048 ms  10.922 ms
```

此处源端到目的端有17跳。注意：有些往返时间看起来非常异常，这可能是路由器太忙不能立即处理分组。

21.3 IGMP

IP协议可用到两种类型的通信：单播和多播。单播是一个发送方和一个接收方之间的通信。它是一对一的通信。但是，有些过程有时需要将同一个报文同时发送给许多的接收方。这称为多播（multicasting），即一到多的通信。多播有许多应用。例如，股票价格的变动要同时通知许多的证券经纪人，或者旅行的取消也会同时通知许多旅行代理，其他一些应用包括远程学习和视频点播等。

因特网组管理协议（Internet Group Management Protocol, IGMP）是其中一个必要的，但不是充分的协议（正如我们将会看到），多播也包含其他的协议。在IP协议中，IGMP是一

个辅助协议。

21.3.1 组管理

对于因特网上的多播，我们需要具有路由多播分组能力的路由器。这些路由器的路由表必须由某些多播路由协议更新，这些多播路由协议将在第22章中讨论。

IGMP不是一个多播路由协议，而是一个管理组成员（group membership）的协议。在任何网络中，都存在有一个或多个多播路由器把多播分组分发给主机或其他的路由器。IGMP协议为多播路由器（multicast router）提供关于连接到网络上的主机（路由器）成员状态的信息。

一个多播路由器每天可以接收到数以千计的属于不同组的多播分组。如果一个路由器不了解主机的成员状态，它就必须广播所有的分组。这造成通信量大量的增加并浪费带宽，一个较好的解决办法是在网络中保存一份组列表，该列表中至少有一个忠实的成员。IGMP帮助多播路由器创建和更新这个表。

IGMP是一个组管理协议，它帮助多播路由器创建和更新与每个路由器接口有关的忠实成员的列表。

21.3.2 IGMP报文

IGMP有两种版本，我们讨论现在的版本是IGMPv2。IGMPv2有三种类型的报文（message）：查询（query）、成员报告（membership report）和离开报告（leave report），其中查询报文（query message）有两种类型：普通的（general）和特殊的（special）（见图21.16）。

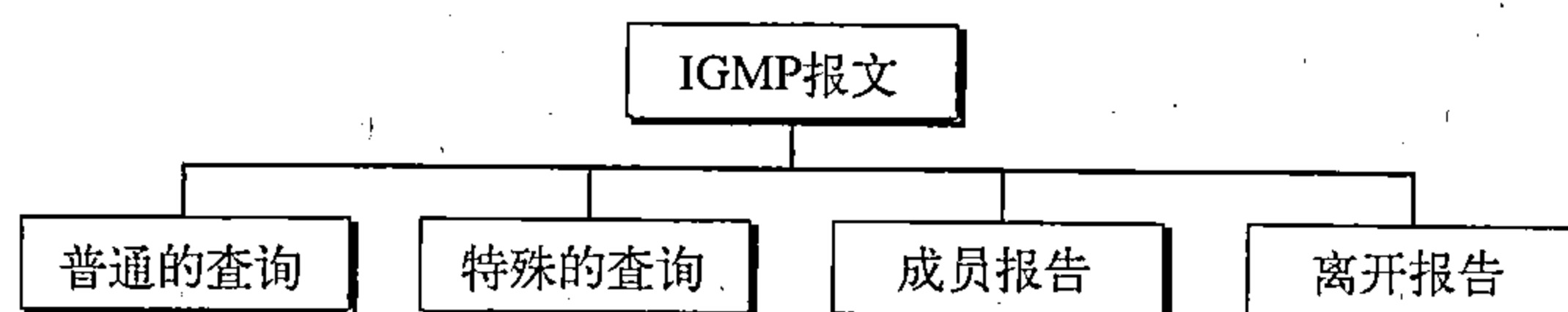


图21.16 IGMP报文类型

21.3.3 报文格式

图21.17显示了IGMP（版本2）报文的格式。

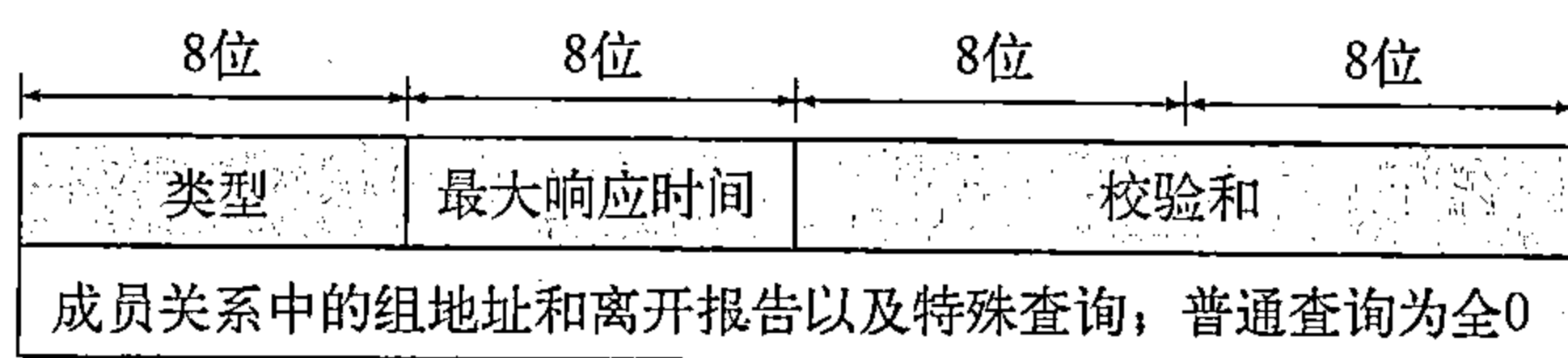


图21.17 IGMP报文格式

- 类型。这个8位字段定义报文类型，如表21.1所示。类型的值用十六进制或二进制标记法表示。
- 最大响应时间。这个8位字段定义回答一个查询所需的时间，该值以十分之一秒为单位。例如，如果其值为100，即是10秒。在查询报文中，该值是非0的；在其余两个类型的报文其值为0。稍后我们会看到它的应用。
- 校验和。这个16位字段是校验和，校验和以8字节的报文计算。

表21.1 IGMP类型字段

类 型	值
普通的或特殊的查询	0x11或00010001
成员报告	0x16或00010110
离开报告	0x17或00010111

- 组地址。对于普通的查询报文，这个字段的值为0，在特殊的查询、组成员报告和离开报告报文中，这个值定义为组标识符（groupid）（组的多播地址）。

21.3.4 IGMP操作

IGMP是本地运行的。连接到网络的多播路由器有一个组多播地址表，该表内至少有一个忠实的成员（见图21.18）。

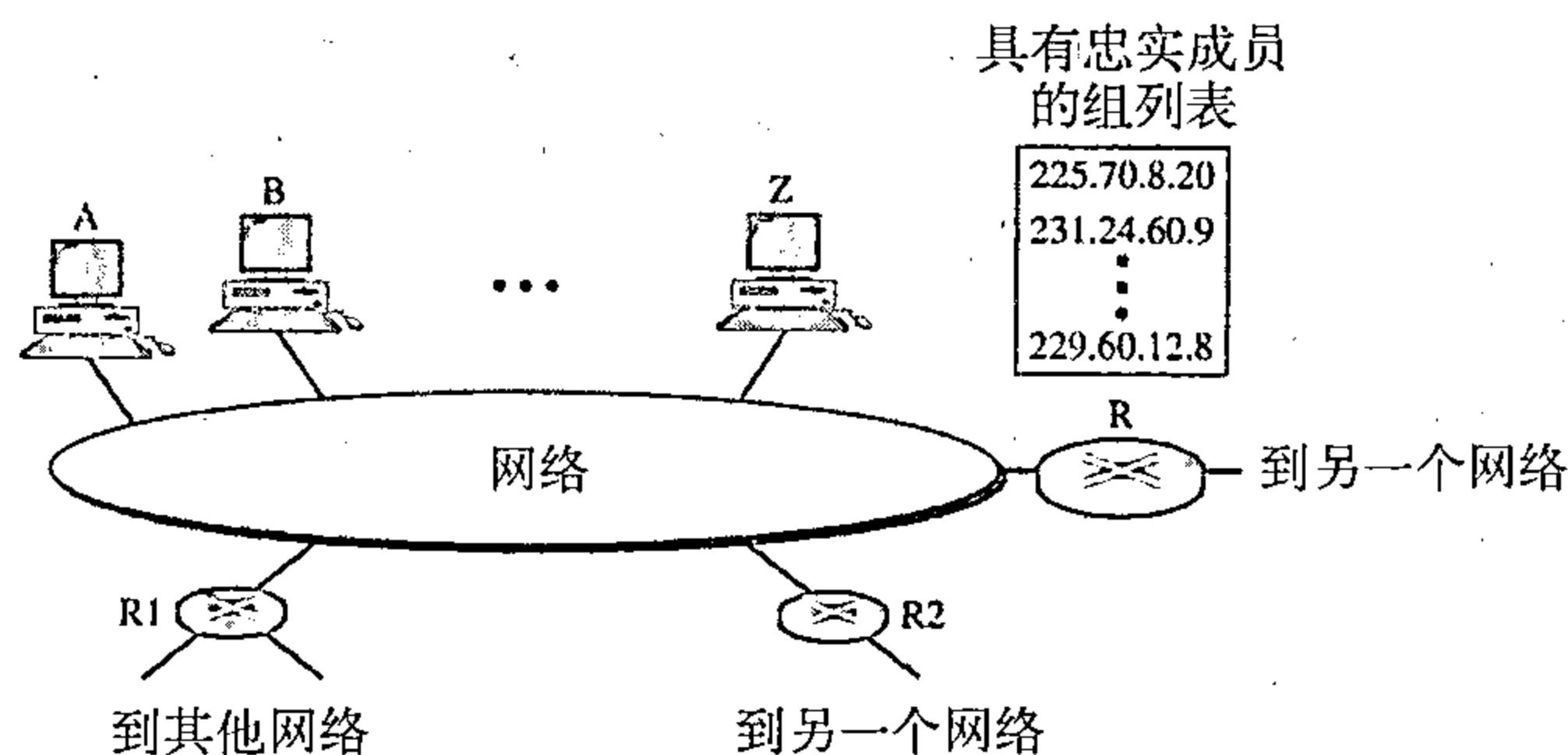


图21.18 IGMP操作

对每一组，都有一个路由器负责将多播分组分发给指定的那个组。这就是说如果与网络相连的多播路由器有三个，它们的组标识符（groupid）都是相互排斥的。例如，在图21.18中只有路由器R分发其多播地址为225.70.8.20的分组。

主机或多播路由器在一个组中可能存在成员关系。当一个主机有成员关系时，就意味着它的一个进程（一个应用程序）接收到来自某一组的多播分组。当一个路由器具有成员关系时，就意味着一个连接到它的其他接口的网络接收到这些多播分组。我们说该主机或路由器对该组有加入请求。在这两种情况下，主机和路由器保存一个组标识符表，并将它们的关系中继给分发路由器。

例如，在图21.18中，路由器R是分发路由器。其余两个多播路由器（R1和R2）依赖于路由器R维护的组列表，它们可能是这个网络中路由器R的接收者。对其他网络的这些组来说，R1和R2可能是分发路由器，而不在本网络上。

加入一组

一个主机或路由器可加入一个组。每一个主机维护一个组内成员进程表。当一个进程要加入到一个新组时，它就向主机发送请求。该主机就在它的表中增加该进程的名字和所请求的组的名字。如果这是在该特殊组中的第一个成员关系的请求，则该主机就向多播路由器发送一个成员关系报告报文。如果这不是第一个成员关系，则不需要发送成员关系报告，因为主机已是组的成员，它已经接收了这个组的多播分组。

协议要求发送两次成员关系报告，在几分钟内一个接着一个地发送。这样，如果第一个丢失或损坏了，第二个可代替它。

在IGMP中，成员关系报告一个接着一个地发送两次。

离开一个组

当主机发现没有进程对一个特殊的组有加入请求时，它就发送一个离开报告。同样的，当路由器发现没有一个与网络连接的接口对一个特殊的组有加入请求，它就发送一个关于该组的离开报告。

但是，当一个多播路由器接收到一个离开报告时，因为报告只是来自一个主机或路由器，

可能还有其他主机或路由器依然存在对该组有加入请求，所以它不可能立即从它的多播地址表中清除该组。为了确保这样做，路由器发送一个特殊查询报文，并插入有关该组的组标识符或多播地址（multicast address）。对任意主机或路由器，路由器允许有一个规定的时间对主机或路由器做出响应。如果在这规定时间内没有接收到加入请求（成员关系报告），那么路由器就假定网络中没有忠实的成员，并从它的列表中清除该组。

监视成员关系

主机或路由器可通过发送成员关系报告报文加入到一个组中，也可通过发送一个离开报告离开一个组。但是，发送这两种类型的报告是不够的。考虑这样的情形，只有一个主机对一个组中有加入请求，但是该主机关闭或从系统中删除。多播路由器将永远接收不到离开报告，这个如何处理？多播路由器负责监视局域网上的所有主机或路由器，以发现它们是否愿意继续它们在一个组中的成员关系。

这个路由器周期性地（默认值为125秒）发送一个普通的查询报文。在这个报文中，组地址字段被设置为0.0.0.0。这就是说查询成员关系的延续需要考虑一个主机所加入的所有的组，而不是仅一个组。

普通查询报文没有定义一个特殊的组。

路由器期待从多播地址表中的每个组得到一个回答，即使一个新组也不例外。查询报文的最大响应时间为10秒（字段中的值为100，而这个是以十分之一秒为单位的）。当主机或路由器接收到普通查询报文时，如果它对一个组有加入请求，它就用一个成员关系报告来响应。但是如果这是一个公用的加入请求（例如，两个主机对同一组有加入请求）时，只有一个响应发送到那个组，以防止不必要的通信量，这称为一个延迟响应。注意：必须仅有一个路由器发送查询报文（通常称为查询路由器），这也是为了防止不必要的通信量。我们在后面讨论这个问题。

延迟响应

为了防止不必要的通信，IGMP使延迟响应策略（delayed response strategy）。当一个主机或路由器接收到查询报文后，它并不立即响应而是经过一定的时间间隔后才发出响应。每一个主机或路由器使用一个随机数建立一个计时器，计时器在1秒到10秒之间。截止时间可以以1秒或更小的时间为步长。对多播组地址表中的每一个组设置一个计时器。例如，计时器对第一个组可用2秒截止，但对第三个组可用5秒截止。每个主机或路由器在它的计时器截止之前一直等待，直到它的计时器截止时间到了才发送一个成员关系报告报文。在等待过程中，如果另一个主机或路由器的计时器对同一个组截止得较早，该主机或路由器就发送一个成员关系报告。因为我们不久就会见到，报告是广播的，正等待的主机或路由器接收到报告而且知道不需要为同一个组发送一个重复报告，这样等待站点就取消它的相应的计时器。

例21.6

设想网络有三个主机，如图21.19所示。

在时刻0，接收到查询报文，每一个组的随机延迟时间（以十分之一秒为单位）在该组地址后面表出，如图所示。试说明其报告序列。

解：

事件发生的序列如下：

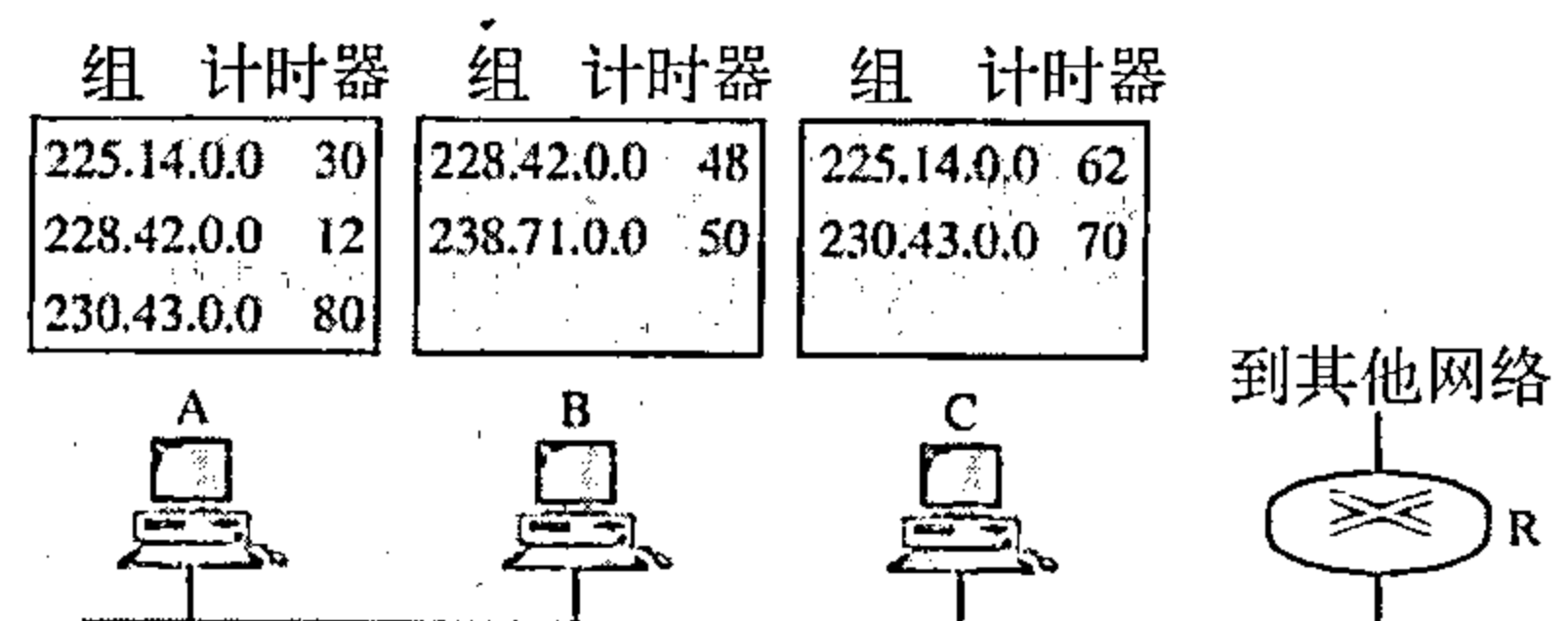


图21.19 例21.6

- a. 时刻12：此时主机A中的组地址228.42.0.0的计时器截止，发送一个成员关系报告。路由器和每个主机（包括主机B）接收到这个报告，这样主机B删除组地址228.42.0.0的计时器；
- b. 时刻30：此时主机A中的组地址225.14.0.0的计时器截止，发送一个成员关系报告。路由器和每个主机（包括主机C）接收到这个报告，这样主机C删除组地址225.14.0.0的计时器；
- c. 时刻50：此时主机B中的组地址238.71.0.0的计时器截止，发送一个成员关系报告。路由器和每个主机接收到这个报告。
- d. 时刻70：此时主机C中组地址230.43.0.0的计时器截止，发送一个成员关系报告。路由器和每个主机（包括主机A）接收到这个报告，主机A删除组地址230.43.0.0的计时器。

注意：如果每个主机为多播地址表中的每个组地址都发送一个报告，则将是7个报告，而用这种策略只发送4个报告。

查询路由器

查询报文可以生成许多响应。为了防止不必要的通信，IGMP为每一个网络设计了一个路由器作为查询路由器（query router）。只有这个路由器发送查询报文，其他路由器都是被动的（它们接收响应并更新它们的列表）。

21.3.5 封装

IGMP报文封装在IP数据报中，而IP数据报本身又封装在帧中，见图21.20所示。

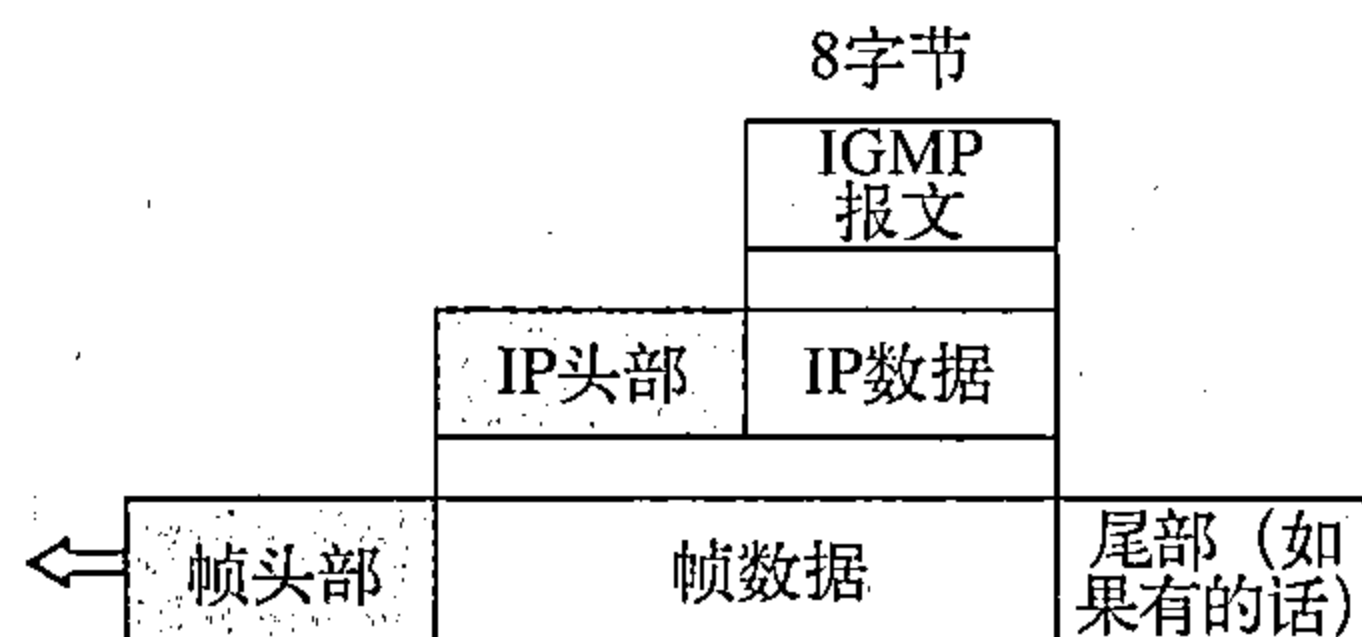


图21.20 IGMP分组的封装

在网络层封装

对于IGMP协议，IP分组的协议字段的值是2。

在协议字段中携带这个值的每一个IP分组都需要给IGMP协议传送数据。当这个报文被封装在IP数据报中时，TTL的值必须为1。这是必需的，因为IGMP的域是局域网，IGMP报文不能传送到局域网之外。这个TTL的值为1保证了报文不能离开这个局域网，因为第一个路由器就将此值减小为0，因而就将其丢弃。表21.2显示了每一种类型报文的IP地址。

携带IGMP分组的IP分组的TTL字段的值为1。

查询报文是用多播地址224.0.0.1进行广播，所有主机和路由器都将接收到该报文。成员关系使用与被报告的多播地址（组标识符）相同的地址进行广播。接收到该分组的每个站点（主机或路由器）可立即确定（从头部）哪一个组报告已经被发送。由于

前面已讨论过，相应的未发送报告的计时器可以被删除。站点不需要打开该分组去寻找组标识符，这个地址被复制在一个分组中，它是报文本身的一部分，也是IP头部一个字段。复制可以防止差错。离开报告报文用多播地址224.0.0.2广播（这个子网上所有的路由器），因此路由器都接收到这类报文，主机也都接收到这类报文，但不理它。

在数据链路层封装

在网络层，IGMP报文封装在IP分组中，作为IP分组处理。但是，因为IP分组有一个多播IP地址，所以ARP协议不能找到在数据链路层转发该分组的对应的MAC（物理）地址。接着发生的事情依赖于下面的数据链路层是否支持物理多播地址。

物理多播地址支持 大多数局域网支持物理多播寻址，以太网就是其中的一种。以太网的物理地址（MAC地址）是6字节（48位）。如果以太网地址的前25位是000000010000000010111100，

表21.2 目的IP地址

类 型	IP目的地址
查询	224.0.0.1这个子网上的所有系统
成员关系报告	组的多播地址
离开报告	224.0.0.2这个子网上的所有路由器